

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
филиал федерального государственного автономного
образовательного учреждения высшего образования
«Мурманский арктический университет» в г. Апатиты
(филиал МАУ в г. Апатиты)

КАФЕДРА ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

МЕЖДИСЦИПЛИНАРНАЯ КУРСОВАЯ РАБОТА

РАЗРАБОТКА КОМПОНЕНТОВ ИНФОРМАЦИОННОЙ СИСТЕМЫ ДЛЯ
ИССЛЕДОВАТЕЛЬСКОГО ПОИСКА В МАССИВАХ СТРУКТУРИРОВАННЫХ ДАННЫХ
ВЗАИМОДЕЙСТВУЮЩИХ С БОЛЬШИМИ ЯЗЫКОВЫМИ МОДЕЛЯМИ

Выполнил обучающийся 4 курса
Фигуркин Даниил Сергеевич
Направление подготовки 09.03.02 Информационные
системы и технологии
Направленность (профиль): Программно-аппаратные
комплексы
очная форма обучения
группа 4БИСИТ-ПАК_АФ

Научный руководитель:
Федоров Андрей Михайлович – канд. техн. наук

г. Апатиты
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
Глава 1. ТЕОРИЯ И АНАЛИТИКА	5
1.1. Структурированные данные и структурированный текст	5
1.2. Исследовательский поиск по коду	6
1.3 Типовые паттерны интеграции в LLM-зависимых приложениях	9
1.4. Проблематика и ограничения	11
1.5 Типовые подходы к организации LLM-зависимых проектов	13
1.6 Формулировка задачи и направления улучшения	14
1.7 Выводы по главе 1	18
ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ И АНАЛОГОВ.....	19
2.1 Обзор выбранных технологий	19
2.2 Аналоги и отличительные особенности разработанного решения.....	22
2.3 Выводы по главе 2.....	27
ГЛАВА 3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ.....	29
3.1. Постановка практической задачи и требования к решению	29
3.2. Общая архитектура разработанного решения.....	31
3.3. Формат правил аннотирования и принцип их применения.....	33
3.4. Алгоритм работы аннотатора и формирование результатов.....	35
3.5. Набор правил и покрываемые точки интеграции LLM.....	37
3.6. Обеспечение воспроизводимости запуска.....	38
3.7. Экспериментальное применение и анализ результатов	39
3.8. Выводы по главе 3.....	42
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	46
ПРИЛОЖЕНИЕ 1. ПРИМЕР ПРОГРАММЫ ДО И ПОСЛЕ ИЗМЕНЕНИЯ	47
ПРИЛОЖЕНИЕ 2. ПРОГРАММА АННОТАТОР	50
ПРИЛОЖЕНИЕ 3. ПРИМЕР ПРАВИЛ АННОТИРОВАНИЯ И ОТЧЁТА	58
ПРИЛОЖЕНИЕ 4. МАТЕРИАЛЫ ДЛЯ ЗАПУСКА	61

ВВЕДЕНИЕ

Широкое распространение приложений, использующих большие языковые модели (LLM) [1, 2], привело к появлению значительного числа учебных и прикладных проектов, реализованных на основе конкретных провайдеров и библиотек. На практике такие проекты нередко оказываются жёстко привязанными к одному API, например OpenAI [3], что обусловлено различиями в способах аутентификации, форматах запросов, работе с контекстом и особенностями используемых библиотек. При изменении условий доступа, политики использования ключей или требований к инфраструктуре возникает необходимость переноса проекта на альтернативные решения, в частности на отечественные модели YandexGPT [5] и GigaChat [4]. При этом перенос зачастую выполняется вручную и требует значительных затрат времени [12].

Актуальность данной работы обусловлена потребностью в подходе, который снижает трудоёмкость первичного анализа и частичной переработки LLM-зависимых приложений при миграции между провайдерами [11]. В рамках курсовой работы рассматривается подход, основанный на оценке сложности преобразования проекта и аннотировании исходного кода по набору правил. Аннотирование позволяет автоматически выделять фрагменты кода, связанные с использованием LLM, формируя набор предлагаемых изменений, а также отчёт о применённых правилах. Оценка сложности, в свою очередь, помогает заранее определить, степень доступности проекта для миграции.

Цель работы — разработать и обосновать подход к снижению трудоёмкости миграции LLM-зависимых приложений на Python на альтернативные провайдеры (YandexGPT и GigaChat) за счёт аннотирования кода по правилам и применения методики оценки сложности преобразования.

Для достижения цели решаются следующие задачи:

- проанализировать особенности интеграции LLM в приложения на Python и выделить типовые точки привязки к провайдеру;

- сформировать требования к разрабатываемому решению;
- разработать формат правил аннотирования и механизм их применения к исходному коду;
- предложить методику оценки сложности преобразования проекта;
- провести экспериментальное применение предложенного подхода.

Объект исследования — приложения на Python, использующие большие языковые модели в прикладных задачах.

Предмет исследования — методы снижения трудоёмкости миграции таких приложений между LLM-провайдерами за счёт аннотирования исходного кода по правилам и оценки сложности преобразования.

Хронологические рамки исследования: в работе рассматриваются подходы, инструменты и open-source проекты в области интеграции LLM, актуальные в период 2020 — начало 2025 гг., поскольку именно в этот период сформировался основной пласт прикладных библиотек и типовых решений для разработки LLM-приложений.

Практическая значимость работы заключается в том, что предложенный подход позволяет структурировать процесс миграции LLM-зависимых приложений.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе рассматриваются теоретические основы интеграции LLM в приложения и формируется методика оценки сложности преобразования. Во второй главе выполняется анализ существующих подходов и инструментов. В третьей главе приводится описание разработанного решения, формата правил, процесса аннотирования и результатов эксперимента.

Глава 1. ТЕОРИЯ И АНАЛИТИКА

1.1. Структурированные данные и структурированный текст

Под структурированным текстом в данной работе понимается текстовая информация, обладающая формализованной внутренней организацией и устойчивыми правилами построения. В отличие от естественно-языкового текста, где структура часто задаётся смыслом и контекстом, структурированный текст описывается формальными правилами и может быть представлен как набор синтаксических конструкций, блоков, зависимостей и повторяемых шаблонов.

В качестве массива структурированных данных в работе рассматривается совокупность программных артефактов, представленных исходным кодом. Объектом анализа выступают файлы Python, а также связанные с ними конфигурации и зависимости. Несмотря на текстовую форму, программный код характеризуется выраженной структурой: он подчиняется правилам синтаксиса, содержит иерархию блоков, связи между модулями и типовые паттерны вызова библиотек. Совокупность этих свойств формирует структурный каркас приложения, благодаря чему исходный код может рассматриваться как разновидность структурированного текста, пригодная для анализа, поиска и преобразования [12, 13]. Streamlit [6] выбран как удобный источник примеров, содержащих как предоставление прямой функциональной доступности примера, так доступ к его исходному коду. При этом выделяемые паттерны интеграции LLM не являются специфичными только для Streamlit.

Для LLM-зависимых приложений структурность проявляется довольно выражено, поскольку интеграция языковой модели зачастую реализуется через повторяющиеся функциональные элементы и стандартные этапы обработки поступающей информации. К типовым элементам относятся:

- импорт библиотек конкретного поставщика LLM;
- получение и хранение ключей доступа;
- создание клиента (инициализация объекта модели);

- формирование запроса и/или истории сообщений;
- вызов модели и получение ответа;
- обработка результата и отображение в пользовательском интерфейсе;
- подключение дополнительных инструментов (поиск, работа с файлами, память, контекст).

Таким образом, рассматриваемый массив программных артефактов представляет собой среду, в которой объектами анализа выступают не отдельные строки, а повторяющиеся шаблоны интеграции LLM, различающиеся деталями реализации в разных open-source решениях. Наличие повторяемых паттернов делает возможным исследовательский поиск характерных фрагментов кода, их извлечение и последующую трансформацию, в том числе для переноса проекта на другую языковую модель.

1.2. Исследовательский поиск по коду

Под исследовательским поиском в рамках работы понимается процесс выявления, сопоставления и анализа закономерностей в массиве исходного кода с целью извлечения информации о типовых способах реализации функциональности, построения обобщений и формирования формализуемых решений для дальнейшей автоматизации. Поскольку в качестве структурированных данных рассматриваются программные артефакты такие как исходный код и конфигурации, исследовательский поиск направлен на обнаружение повторяемых фрагментов и архитектурных закономерностей, отражающих реализацию сходных функций в различных проектах.

В данной курсовой работе исследовательский поиск применяется к массиву LLM-зависимых open-source проектов и ориентирован на выделение типовых паттернов интеграции больших языковых моделей, а также на анализ различий, возникающих при использовании разных поставщиков моделей и библиотек. Практически это означает поиск в коде устойчивых конструкций,

связанных с жизненным циклом запроса к LLM и обработкой результата. К таким конструкциям относятся:

- подключение библиотек и создание клиента (инициализация объекта модели);
- получение и хранение ключей доступа, конфигурация параметров модели;
- формирование запроса и структуры сообщений;
- вызов модели и получение ответа (различия форматов ответа и способов доступа к тексту);
- обработка результата и отображение в пользовательском интерфейсе;
- организация памяти/истории диалога;
- использование внешних инструментов (поиск в интернете, работа с файлами, контекст);
- особенности UI-слоя (Streamlit-компоненты, вывод сообщений, обработка событий).

Существуют различные уровни и подходы к поиску по коду:

- строковый поиск (по подстроке): является самым простым, но наименее надёжным методом. Он чувствителен к форматированию, отличиям в стиле кода и вариантам записи выражений. Например, один и тот же смысловой элемент может быть записан в разных проектах по-разному: с различным расположением аргументов, переносами строк, промежуточными переменными и т.п.
- поиск на основе регулярных выражений: позволяет задавать более гибкие шаблоны, однако остаётся зависимым от текстового представления и может давать ложные совпадения, например при наличии строковых литералов, комментариев или совпадений с другими контекстами. Дополнительно усложняется поддержка правил при росте их количества.
- лексический поиск на уровне токенов: использует представление кода как последовательности токенов [8] (ключевых слов, идентификаторов, операторов и т.д.). Такой подход снижает зависимость от форматирования и переносов строк, позволяя сравнивать фрагменты на уровне синтаксических

элементов, а не на уровне символов. Это делает поиск более устойчивым к типичным различиям оформления и стилям программирования.

- синтаксический поиск (AST-подход): опирается на дерево разбора программы и позволяет наиболее строго определять структуры кода. Данный подход обеспечивает высокую точность, однако требует более сложной реализации и аккуратного учёта множества конструкций языка.

Для повышения воспроизводимости и последующей автоматизации результаты исследовательского поиска фиксируются в форме обобщённых шаблонов, которые далее используются при построении набора правил преобразования кода. Таким образом, цель поиска состоит не только в нахождении похожего, но и в формировании практического основания для ответов на вопросы: какие элементы интеграции являются обязательными, какие зависят от конкретного API, какие фрагменты поддаются автоматической трансформации, а какие требуют ручной доработки.

Процесс исследовательского поиска в работе может быть представлен как последовательность этапов:

- отбор проектов и первичная каталогизация (название, назначение, используемые библиотеки, наличие памяти, инструментов и др.).
- выделение точек интеграции LLM в каждом проекте (места формирования запроса, вызова модели, обработки ответа).
- сопоставление реализаций одинаковых функций в разных проектах (например, “чат без памяти”, “чат с историей”, “вопрос-ответ по документам” и т.п.).
- обобщение найденных решений и формирование каркаса интеграции, независимого от конкретного поставщика модели.
- формализация в виде правил преобразования (структуры “найти/заменить”), применимых к исходному коду при миграции проекта на другую LLM.

Выделение и фиксация таких паттернов позволяет переносить полезные LLM-зависимые прототипы на альтернативные модели без полной переработки

проекта, что особенно важно в условиях ограниченной доступности отдельных платформ и различий в интерфейсах взаимодействия с LLM.

1.3 Типовые паттерны интеграции в LLM-зависимых приложениях

Интеграция больших языковых моделей в прикладные приложения, как правило, строится вокруг повторяемых архитектурных решений, которые можно рассматривать как паттерны. Под паттерном в данной работе понимается устойчивый способ организации взаимодействия приложения с LLM, охватывающий основные этапы жизненного цикла запроса: от подготовки входных данных и контекста до получения ответа и его представления пользователю. Наличие повторяемых элементов позволяет использовать их как основу для исследовательского поиска по массивам структурированного текста (исходного кода), а также как базу для последующей стандартизации и частичной автоматизации переноса проектов между различными поставщиками моделей.

Ниже приведены основные паттерны интеграции LLM, встречающиеся в LLM-зависимых приложениях:

а) паттерн подключения и конфигурирования модели: первый типовой паттерн связан с тем, как приложение подключает языковую модель и задаёт параметры её работы. На практике он включает выбор библиотек и зависимостей, импорт модулей, создание объекта клиента и настройку параметров генерации (например, температура, максимальная длина ответа, режим выдачи результата и др.). Для задач миграции данный паттерн является базовым, так как при смене провайдера меняются используемые классы, способы инициализации и набор поддерживаемых параметров.

б) паттерн управления доступом и параметрами подключения: LLM-интеграция практически всегда предполагает наличие конфиденциальных параметров: ключей доступа, идентификаторов, URI модели, области применения и иных настроек. В прототипах часто встречается ввод таких параметров через интерфейс приложения либо их загрузка из переменных

окружения и конфигурационных файлов. С точки зрения переносимости этот паттерн является критическим: при смене провайдера изменяются требуемые поля, правила их проверки и способ передачи в клиент. Соответственно, обнаружение и фиксация этих участков кода являются важной задачей для аннотирования и оценки сложности преобразования.

в) паттерн формирования запроса и контекст: Следующий этап жизненного цикла — формирование запроса к модели. Даже в простых сценариях запрос включает пользовательский ввод и дополнительные инструкции, например, системные установки или шаблоны промпта. В более развитых приложениях к запросу добавляется внешний контекст: текст документа, результаты поиска, извлечённые фрагменты, параметры задач и т.п. Типовой особенностью является необходимость поддерживать согласованность между интерфейсом приложения и тем, в каком виде запрос передаётся в модель (строка, структура сообщений, последовательность ролей). В рамках исследовательского поиска важно фиксировать, какие части запроса определяются логикой приложения, а какие обусловлены форматом конкретного API, поскольку это разделение упрощает последующую миграцию.

г) Паттерн вызова модели и получения результата: после формирования запроса выполняется обращение к LLM и получение результата. На концептуальном уровне этот шаг одинаков, сначала происходит выполнение запроса, после чего следует получение ответа. На уровне реализации наблюдаются различия в механизмах вызова: методы, параметры, поддержка потоковой генерации, а также в форме представления результата.

д) Паттерн обработки результата и представления пользователю: большинство LLM-прототипов ориентированы на пользовательский диалог или на получение сформированного ответа. Поэтому важной частью является обработка результата: очистка/постобработка текста, форматирование, вывод в интерфейсе, сохранение в историю или запись в файл. В приложениях с графическим интерфейсом добавляется паттерн визуального представления диалога: разделение ролей на пользователя и ассистента, отображение блоков

сообщений, логика кнопок и событий. Именно на этом этапе проявляется различие между простыми проектами: линейный вывод ответа и сложными: кастомный вывод, дополнительные режимы отображения, разветвлённая логика взаимодействия.

е) Паттерн использования внешних инструментов и источников данных: в более сложных приложениях LLM используется не изолированно, а как компонент, дополняемый инструментами: поиском, загрузкой документов, извлечением контекста, обращением к внешним данным [13]. Такие проекты обычно включают дополнительные шаги: подготовку входных данных, преобразование форматов, хранение промежуточных результатов. Наличие инструментов существенно усложняет переносимость: возрастает число зависимостей и точек интеграции, а также вероятность несовместимости отдельных библиотек при смене провайдера или платформы. С точки зрения исследовательского поиска этот паттерн является одним из ключевых факторов для классификации проекта по сложности преобразования.

ж) Паттерн устойчивости и обработки ошибок: даже в прототипах распространены проверки корректности конфигурации: наличие ключей, доступность модели, обработка исключений и сообщения пользователю о проблемах. Этот паттерн важен для воспроизводимости экспериментов: при множестве проектов единообразное представление ошибок ускоряет диагностику и снижает трудозатраты при сравнительном тестировании.

1.4. Проблематика и ограничения

Несмотря на высокую прикладную ценность современных больших языковых моделей, их использование в реальных задачах разработки и исследовательского прототипирования сопряжено с рядом ограничений, напрямую влияющих на универсальность и переносимость программных решений:

а) Во-первых, существенным барьером является доступность вычислительных сервисов и API: значительная доля open-source примеров и

демонстрационных приложений ориентирована на использование коммерческих зарубежных платформ (например, OpenAI). В условиях ограничений доступа, высокой стоимости или невозможности получения ключей такие проекты становятся трудновоспроизводимыми и непригодными для учебной и исследовательской практики. Следствием является снижение ценности готовых прототипов: при отсутствии доступа к исходному провайдеру модельного API проект фактически теряет работоспособность без переработки.

б) Во-вторых, переносимость осложняется неоднородностью интерфейсов и библиотек для работы с LLM. Даже при одинаковой общей логике запрос-ответ разные поставщики и программные обвязки используют различные способы:

- задания параметров и аутентификации;
- формирования структуры сообщений и истории диалога;
- организации памяти, контекста и инструментов (поиск, работа с документами, RAG-компоненты).

Это приводит к тому, что миграция проекта между LLM-провайдерами зачастую требует комплексной переработки нескольких фрагментов кода, а также согласования зависимостей и версий библиотек. Для начинающих разработчиков такая фрагментированность повышает порог входа и увеличивает время, необходимое для практической работы с прототипами.

в) В-третьих, при работе с набором LLM-зависимых проектов возникают повторяющиеся организационно-технические операции, которые приходится заново выполнять для каждого прототипа. К ним относятся настройка окружения, установка зависимостей, конфигурация ключей доступа, запуск приложения, проверка работоспособности и сравнение поведения разных реализаций. При увеличении числа проектов это приводит к существенным затратам времени и затрудняет исследовательский анализ архитектурных решений.

Указанные ограничения обосновывают необходимость создания инфраструктуры, ориентированной на ускорение исследования и эксплуатации LLM-зависимых прототипов. Такая инфраструктура должна обеспечивать:

- воспроизводимое развертывание проектов (управление окружением и зависимостями);
- унификацию и автоматизацию адаптации проектов при замене поставщика LLM;
- снижение повторяемости ручных операций (ввод и проверка ключей, подготовка конфигураций, типовые проверки работоспособности).

Таким образом, стандартизация подходов к преобразованию и запуску LLM-проектов повышает продуктивность исследовательского процесса, улучшает воспроизводимость результатов и снижает трудоёмкость переноса приложений между различными LLM-платформами.

1.5 Типовые подходы к организации LLM-зависимых проектов

Большинство решений ориентировано на ускорение разработки новых приложений и удобство экспериментов, тогда как перенос уже созданного проекта между разными поставщиками моделей часто требует отдельной работы с исходным кодом и окружением. В исследовательской и учебной практике это приводит к ситуации, когда часть задач закрывается готовыми средствами, но ключевые действия по миграции и проверке остаются на стороне разработчика.

На практике можно выделить несколько распространённых подходов к организации LLM-зависимых проектов:

а) разработка под конкретного провайдера: проект строится на официальной или наиболее распространённой библиотеке поставщика. Подход удобен для быстрого старта, но приводит к сильной связанности с API и форматом ответа, из-за чего при смене провайдера требуется переработка кода.

б) использование унификации и адаптеров: для снижения зависимости вводится единый интерфейс обращения к модели, а конкретные провайдеры подключаются через обёртки. Подход улучшает переносимость, однако обычно

требует, чтобы проект изначально проектировался с этим слоем, либо предполагает рефакторинг существующего кода.

в) шлюзы и прокси: между приложением и моделью добавляется прослойка, обеспечивающая единый входной интерфейс и маршрутизацию запросов. Такой подход может уменьшать зависимость на уровне протокола, но не отменяет необходимости анализа исходников, если проект использует специфические вызовы SDK или нестандартную обработку ответа.

г) ручная миграция и сопровождение: наиболее распространённый сценарий в условиях множества разнородных прототипов: по отдельности изменяются импорты, параметры доступа, вызовы модели и зависимости, после чего выполняется проверка запуска. Подход гибкий, но при росте числа проектов быстро становится трудоёмким и плохо масштабируется.

Таким образом, современное состояние практик характеризуется высокой вариативностью, однако для сценария миграции массива проектов сохраняется общий недостаток: отсутствует системный механизм, который одновременно облегчает поиск точек интеграции в коде и позволяет заранее оценивать трудоёмкость переноса.

1.6 Формулировка задачи и направления улучшения

Анализ типовых паттернов интеграции больших языковых моделей и рассмотрение существующих подходов показывают, что основная трудоёмкость миграции LLM-зависимых приложений связана с двумя факторами. Во-первых, требуется выявить в исходном коде все участки, завязанные на конкретного поставщика модели: подключение библиотек, ввод и проверку параметров доступа, формирование запроса, вызов модели, извлечение текста ответа, обработку ошибок и вывод результата. Во-вторых, необходимо заранее понимать ожидаемый объём работ и риски несовместимости, поскольку разные проекты существенно отличаются по числу точек интеграции и используемым возможностям библиотек.

В связи с этим в работе формулируется задача разработки подхода, который снижает трудозатраты на первичный анализ и подготовку проекта к переносу на альтернативного поставщика модели за счёт предварительной оценки сложности преобразования и аннотирования кода.

На практике имеется массив проектов, реализующих сходные пользовательские сценарии (например, диалоговый интерфейс, режим вопрос-ответ, обработка документа), но отличающихся деталями интеграции LLM и используемыми библиотеками. Требуется обеспечить:

- исследовательский поиск по исходному коду, выявление повторяющихся паттернов интеграции;
- каталогизацию и ранжирование проектов по сложности переноса, выделение проектов, допускающих преобразование с минимальными доработками, проектов, требующих значительных доработок, и проектов, перенос которых нецелесообразен в заданных условиях;
- частичную автоматизацию обработки типовых фрагментов кода при замене одного поставщика модели на другой, а также формирование подсказок для ручной доработки;
- воспроизводимый запуск и тестирование преобразованных проектов.

В существующей практике решение часто сводится к ручной миграции: для каждого проекта отдельно заменяются ключевые фрагменты, настраивается окружение и проверяется результат. Подход имеет преимущества в виде точечного контроля качества, однако при работе с массивом проектов его недостатки становятся критичными: высокая трудоёмкость, повторяемость однотипных действий, риск ошибок и зависимость результата от локального окружения. В рамках данной работы предлагается улучшить процесс за счёт построения инфраструктуры, включающей следующие принципы и компоненты:

- Стандартизация и фиксация паттернов интеграции: повторяющиеся элементы рассматриваются как структурные признаки, подлежащие выявлению и описанию.

– Формализация преобразований в виде набора правил: типовые изменения фиксируются как структуры “что найти / чем дополнить”, что позволяет накапливать и переиспользовать базу знаний о миграции проектов.

– Каталогизация и ранжирование проектов: вводится классификация переносимости, позволяющая заранее оценивать объём работ и выбирать оптимальный путь адаптации.

– Воспроизводимое развертывание: использование единых сценариев запуска снижает влияние различий окружения и ускоряет проверку результатов преобразования.

Для снижения трудоёмкости на этапе анализа вводится методика оценки сложности преобразования проекта. Оценка строится на признаках, которые могут быть выявлены в исходном коде и отражают насыщенность проекта точками интеграции LLM и внешними зависимостями. Каждому признаку присваивается значение от 0 до 2 баллов, после чего вычисляется суммарная оценка.

Таблица 1.1. – Бальные критерии оценки сложности преобразования LLM-зависимого проекта

Признак	0 баллов	1 балл	2 балла
Способ обращения к модели	один запрос, один ответ	диалог с историей сообщений	сложная логика диалога, несколько режимов вызова
Управление состоянием	Состояния нет	локальная память без сложной логики	внешнее хранение, суммаризация, сложная логика контекста
Внешние источники данных	отсутствуют	один источник (например, файлы или web-поиск)	несколько источников и этапов подготовки данных
Зависимость от сторонних библиотек	Только SDK провайдера или прямой клиент	используется дополнительная обвязка	используется фреймворк с цепочками и инструментами
Представление результата / UI	линейный вывод ответа	форматирование и базовая структура вывода	разветвлённый вывод, несколько режимов отображения
Нестандартные компоненты и зависимости	отсутствуют	присутствуют, но ограничено	множество или нестабильные зависимости

Суммарная оценка используется для предварительного ранжирования проектов. При этом высокий балл означает увеличение объёма работ и риск несовместимости при миграции.

Для принятия практических решений вводится классификация проектов по переносимости. Классификация основывается на результатах оценки, а также на наличии ограничений, препятствующих переносу.

Таблица 1.2. – Категории проектов по сложности и переносимости

Категория	Баллы	Особенности	Рекомендуемая стратегия
Простой	0–3	Минимальная логика, мало интеграций, один сценарий	Универсальное преобразование правилами + быстрая проверка запуска
Сложный	4–7	Несколько интеграций/источников/состояние/форматирование	Правила + ручная доработка по отчёту; возможно расширение набора правил
Под вопросом	8–12 или недостаточно данных	Неоднозначная логика, много “хрупких” мест, трудно классифицировать без просмотра	Дополнительный аудит кода (выявить узлы интеграции) → решить: упростить/переписать часть/перевести в “Сложный” или “Не преобразуемый”
Не преобразуемый	стоп-условия	Несовместимые зависимости/архитектура/требования	Исключить из потока; оформлять как отдельный проект миграции “с переработкой”

Категория “не преобразуемый” определяется наличием ограничений, при которых перенос нецелесообразен. Например, если проект критически зависит от специфичных функций библиотеки, не имеющих устойчивых аналогов в целевом окружении, или если требуемая функциональность не может быть воспроизведена без существенной переработки архитектуры.

Таким образом, направления улучшения по сравнению с распространёнными подходами заключаются в переходе от единичной ручной миграции к системной работе с массивом проектов: от переписывания каждого приложения — к воспроизводимому процессу, сочетающему исследовательский анализ, оценку сложности, аннотирование и частичную автоматизацию обработки типовых фрагментов кода. Это создаёт основу для последующего проектирования компонентов решения и реализации практической части работы.

1.7 Выводы по главе 1

В первой главе были рассмотрены теоретические основания, необходимые для построения подхода к работе с массивом LLM-зависимых проектов. Показано, что исходный код и связанные с ним конфигурации могут рассматриваться как структурированный текст, пригодный для исследовательского поиска, выделения повторяемых элементов и последующей формализации преобразований.

В рамках исследовательского поиска по коду выделены типовые паттерны интеграции LLM, описывающие жизненный цикл запроса: подключение и конфигурирование модели, управление параметрами доступа, формирование запроса и контекста, вызов модели и получение результата, обработка и представление ответа, использование внешних инструментов, а также обеспечение устойчивости и обработка ошибок. Установлено, что именно эти паттерны формируют основные точки привязки приложения к конкретному поставщику модели и определяют объём изменений при миграции.

Рассмотренная проблематика миграции показывает, что перенос LLM-зависимых проектов осложняется недоступностью или высокой стоимостью отдельных API, различиями библиотек и форматов ответа, конфликтами зависимостей и трудностями воспроизводимого развёртывания. Следовательно, при обработке множества прототипов требуется подход, который одновременно снижает трудозатраты анализа, повышает воспроизводимость запуска и уменьшает риск пропуска критичных фрагментов кода.

На основе выявленных особенностей сформулирована задача разработки подхода, объединяющего аннотирование кода по формализованным правилам и предварительную оценку сложности преобразования проекта. Предложена методика бальной оценки и классификация проектов по категориям переносимости, что позволяет заранее ранжировать проекты и выбирать рациональный способ дальнейшей работы.

ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ И АНАЛОГОВ

2.1 Обзор выбранных технологий

В рамках работы был выбран набор технологий, обеспечивающий быстрый цикл прототипирования, воспроизводимость экспериментов и возможность масштабирования решения на массив разнородных LLM-зависимых проектов. Ключевыми критериями выбора являлись: распространённость инструментов и наличие устойчивой практики применения в открытых репозиториях, простота интеграции, совместимость с учебно-исследовательскими задачами, а также возможность воспроизводимого развёртывания и запуска проектов:

а) среда выполнения и разработки:

В качестве базовых платформ выполнения и разработки использованы операционные системы Linux и Windows. Применение двух операционных систем отражает типовой сценарий учебной и практической работы. В рамках выполнения курсовой основная часть проектов разрабатывалась и запускалась в среде Linux, а Windows использовалась для проверки независимости развёртывания и выявления возможных проблем, связанных с окружением и зависимостями.

В качестве основной среды разработки используется Visual Studio Code, так как она обеспечивает удобную навигацию по проектам и поддерживает Python-экосистему, включая работу с виртуальными окружениями, контейнерами и репозиториями. Это особенно важно при анализе массива проектов и выполнении серии однотипных преобразований над несколькими файлами.

б) язык и формат представления правил преобразования:

Для реализации утилит преобразования выбран Python. Он широко применяется для прототипирования, разработки вспомогательных скриптов, интеграции с LLM и создания демонстрационных приложений. Для рассматриваемого класса проектов Python обеспечивает высокую скорость разработки, богатую библиотечную базу и совместимость с большим числом

открытых примеров, что упрощает исследовательский анализ и воспроизведение экспериментов.

Правила преобразования и аннотирования представлены в формате JSON. Формат выбран как человекочитаемый и стандартизированный способ хранения структур вида “найти / вставить”. JSON позволяет хранить расширяемый набор правил как отдельный переносимый артефакт и обеспечивать воспроизводимость преобразования за счёт фиксированного набора правил.

в) инструменты взаимодействия с большими языковыми моделями:

В качестве целевых LLM-платформ рассматриваются отечественные решения YandexGPT и GigaChat, что связано с практической задачей снижения зависимости от зарубежных API и повышения доступности экспериментов в условиях ограничений. Для интеграции применяются библиотеки соответствующих поставщиков или совместимые обвязки, позволяющие выполнять типовые операции: конфигурирование доступа, формирование запроса и получение результата. При этом различия в API и способах обращения к модели рассматриваются как один из факторов, определяющих сложность преобразования проектов.

г) средства обеспечения воспроизводимости:

При исследовательской работе с набором прототипов существенное значение имеет воспроизводимость: возможность повторно запустить приложение в одинаковых условиях, получить сопоставимый результат и снизить влияние различий локальных сред. На практике воспроизводимость нарушается из-за расхождений в версии интерпретатора, наборах библиотек, системных зависимостях и настройках окружения. Для LLM-зависимых приложений проблема усиливается тем, что проекты часто включают значительное число сторонних библиотек и могут быть чувствительны к их версиям.

Одним из распространённых подходов к повышению воспроизводимости является контейнеризация. Контейнер фиксирует среду исполнения приложения (версию языка, набор библиотек, системные зависимости) и тем самым

уменьшает влияние различий между машинами разработчика и тестовой средой. Контейнерный подход особенно полезен в ситуациях, когда требуется воспроизводить запуск прототипа без глобального обновления библиотечной базы на устройстве.

В рамках данной курсовой работы Docker [7] рассматривается как инструмент, обеспечивающий фиксирование окружения и единообразный запуск как утилитарных скриптов, так и демонстрационных LLM-прототипов. Использование контейнеризации позволяет отделить исследуемую логику проектов и результаты преобразования от особенностей локальной установки и тем самым повысить воспроизводимость экспериментов.

д) Управление исходным кодом и артефактами:

GitLab используется как система контроля версий и репозиторий проектов. Это обеспечивает фиксацию изменений, возможность сравнения версий “до/после” преобразования, документирование найденных паттернов и ведение каталога проектов с классификацией сложности их переноса.

В качестве вспомогательного канала обмена артефактами может применяться облачное хранилище, однако ключевые результаты (исходники, правила, отчёты преобразования) целесообразно фиксировать именно в репозитории как основном источнике версий.

е) Средство демонстрации и проверки преобразованного проекта:

Streamlit выбран как основная платформа для создания демонстрационных веб-интерфейсов, поскольку позволяет быстро разрабатывать прототипы без отдельной фронтенд-разработки и широко используется в open-source примерах LLM-приложений. Это делает Streamlit удобной основой для исследования и сравнительного тестирования прототипов, а также для валидации корректности преобразования проектов с точки зрения пользовательского сценария.

Таким образом, выбранный технологический стек соответствует целям работы:

- Python и JSON обеспечивают реализацию и расширяемость правил преобразования;

- Docker и единые сценарии запуска повышают воспроизводимость экспериментов;
- репозиторий фиксирует артефакты и результаты преобразований;
- Streamlit служит удобной основой для апробации и демонстрации результатов на наборе прототипов.

2.2 Аналоги и отличительные особенности разработанного решения

Задача переноса LLM-зависимых приложений на альтернативные платформы частично пересекается с рядом направлений, уже представленных в современном программном инструментарии. Однако большинство существующих решений ориентировано либо на упрощение разработки новых LLM-приложений, либо на унификацию вызовов моделей во время выполнения, тогда как в рамках данной работы акцент сделан на снижение трудоёмкости первичного анализа и подготовки к миграции существующих проектов. Ниже рассмотрены основные группы аналогов и проведено сравнение с разработанным подходом.

а) Инструменты унифицированного доступа к LLM:

К первой группе относятся решения, предлагающие единый интерфейс обращения к различным моделям и поставщикам. Их общая идея заключается в том, что приложение не обращается напрямую к конкретному поставщику, а использует унифицированный слой, который скрывает различия интерфейсов и обеспечивает переключение между моделями. Например, LangChain предлагает стандартные интерфейсы моделей, упрощающие эксперименты и переключение между разными интеграциями.

Сильные стороны подхода:

- снижение зависимости от конкретного поставщика при проектировании новых приложений;
- упрощение переключения между моделями на уровне конфигурации;
- удобство тестирования разных моделей в рамках одной логики приложения.

Ограничения в контексте миграции уже написанного кода:

- если проект изначально не строился на унифицированном интерфейсе, внедрение такого слоя требует рефакторинга и ручного анализа;
- специфические возможности отдельных библиотек часто не переносятся в общий интерфейс без потерь или усложнений;
- инструмент решает задачу как вызывать модель, но не отвечает на вопрос количества усилий для переноса конкретного проекта.

б) Фреймворки оркестрации и сборки LLM-приложений:

Ко второй группе относятся библиотеки, предоставляющие компоненты для построения приложений: управление сообщениями, шаблоны запросов, цепочки обработки, подключение инструментов, работа с внешними источниками данных. Их роль — ускорять разработку и расширять функциональность LLM-приложений.

Сильные стороны подхода:

- ускорение разработки за счёт готовых компонентов;
- унификация типовых сценариев (например, диалог, обработка документов, подключение инструментов);
- более структурная организация приложения по сравнению с ручной интеграцией.

Ограничения в контексте миграции уже написанного кода:

- переносимость может ухудшаться из-за зависимости от конкретного фреймворка, его версий и набора интеграций;
- применение фреймворка не устраняет необходимость анализа существующего кода, если приложение уже написано иначе;
- при несовместимости отдельных компонентов миграция может потребовать не замены вызовов, а переработки логики.

в) Шлюзы, прокси и средства мониторинга LLM-вызовов:

Существуют решения, фокусирующиеся на эксплуатации: промежуточные сервисы для маршрутизации запросов, учёта затрат, логирования, контроля

качества и безопасности. Они полезны для наблюдаемости и централизованного управления обращениями к моделям.

Сильные стороны подхода:

- улучшение контроля и мониторинга вызовов модели;
- удобство для командной разработки и эксплуатации;
- централизованная настройка политик и ограничений.

Ограничения в контексте миграции уже написанного кода:

- такие решения работают “вокруг” приложения и не решают задачу поиска и переработки участков кода;
- наличие прокси не устраняет различия библиотек и форматов на уровне исходников;
- не даётся методика оценки сложности переноса проекта по структуре его кода.

г) Инструменты анализа и преобразования исходного кода:

К отдельной группе относятся средства, предназначенные для анализа исходного кода и выполнения преобразований. В эту группу входят парсеры, инструменты рефакторинга и преобразования программ, а также подходы, основанные на синтаксическом анализе. LibCST предоставляет lossless CST-представление Python-кода и используется для построения codemod-инструментов и автоматизированного рефакторинга. Bowler [10] позиционируется как инструмент рефакторинга на уровне синтаксического дерева и опирается на стек, происходящий от lib2to3 [9]. Важно учитывать, что модуль lib2to3 удалён в Python 3.13, поэтому применение подобных решений может требовать ограничений по версии интерпретатора или использования совместимых замен.

Сильные стороны подхода:

- высокая точность при корректно выбранном уровне анализа;
- возможность формализовать преобразования и применять их повторно;
- пригодность для построения правил, связанных со структурой кода.

Ограничения в контексте миграции уже написанного кода:

- универсальные инструменты преобразования кода не учитывают предметную специфику LLM-интеграции и не предоставляют готовой классификации переносимости;

- разработка полного преобразователя под все варианты интеграции требует значительных трудозатрат и быстро усложняется при росте числа поддерживаемых стилей кода и библиотек.

д) Отличительные особенности разработанного решения:

Разработанный в рамках курсовой работы подход отличается от рассмотренных аналогов тем, что он ориентирован не на создание нового приложения и не на инфраструктуру эксплуатации, а на организацию процесса миграции массива существующих LLM-зависимых проектов и снижение трудоёмкости первичного этапа работ. Отличительные особенности можно сформулировать следующим образом:

- Фокус на массиве проектов как объекте исследования: в отличие от решений, предлагающих унифицированный интерфейс, в данной работе основной результат нацелен на ситуацию, когда уже существует проект со своей логикой и стилем, и требуется быстро определить, где в коде находятся точки привязки к провайдеру и насколько сложен перенос.

- Аннотирование исходного кода по формализованным правилам: предлагается подход, при котором найденные паттерны интеграции фиксируются в виде правил и применяются к исходному коду для получения размеченной версии. Это превращает поиск по проекту в воспроизводимую процедуру и формирует карту мест, требующих внимания при переносе.

- Методика оценки сложности преобразования и ранжирование проектов: в отличие от большинства аналогов, в работе вводится формализованная оценка сложности, позволяющая заранее ранжировать проекты по переносимости и выбрать рациональную стратегию работы с каждым проектом: быстрое преобразование, преобразование с доработками, дополнительный анализ или отказ от переноса при наличии ограничивающих условий.

– Ориентация на отечественные целевые платформы: поскольку целевыми поставщиками рассматриваются YandexGPT и GigaChat, различия интерфейсов и требований к доступу учитываются как фактор сложности. Это позволяет анализировать переносимость не абстрактно, а относительно конкретных целевых условий и ограничений.

– Поддержка воспроизводимости экспериментов через единообразное развёртывание: в работе отдельно учитывается аспект повторяемости запуска прототипов, что важно при обработке множества проектов и проверке результатов преобразования. Воспроизводимость рассматривается как обязательное условие корректного сравнения и валидации.

Помимо позиционирования относительно аналогов важно уточнить, какие именно участки кода рассматриваются в работе как наиболее значимые для миграции. Независимо от конкретной библиотеки, интеграция LLM в прикладном проекте обычно концентрируется в нескольких повторяемых узлах, где различия между провайдерами проявляются наиболее заметно и где чаще всего требуется ручная переработка при переносе. Эти узлы используются как практическая основа для набора правил аннотирования и для оценки сложности преобразования проектов.

К таким узлам относятся:

- ввод и проверка параметров доступа к модели;
- подключение библиотек и создание клиента;
- конфигурирование параметров генерации;
- формирование запроса и структуры сообщений;
- вызов модели и извлечение текста ответа;
- обработка ошибок и информирование пользователя;
- вывод результата и обновление состояния приложения (например, история сообщений).

Таким образом, разработанное решение занимает отдельную нишу относительно существующих инструментов: оно не заменяет фреймворки разработки и не является инфраструктурой эксплуатации, а служит средством

снижения ручного труда при миграции за счёт объединения исследовательского поиска по коду, правил аннотирования и методики оценки сложности преобразования.

2.3 Выводы по главе 2

Во второй главе был рассмотрен набор технологий, используемых для исследования и последующей апробации подхода к миграции LLM-зависимых проектов. Показано, что выбранный стек обеспечивает условия для воспроизводимой работы с массивом прототипов: разработку и запуск в среде Linux с дополнительной проверкой переносимости в Windows, применение Python как основного языка обработки исходного кода, хранение правил преобразования в формате JSON и использование системы контроля версий для фиксации изменений и сравнения версий “до/после”.

Проведён обзор групп аналогов, связанных с применением больших языковых моделей. Отмечено, что большинство существующих решений ориентировано на разработку новых LLM-приложений или на унификацию вызовов моделей во время выполнения, тогда как задача миграции уже существующих проектов требует отдельного внимания к поиску точек интеграции в исходном коде и снижению доли ручной работы на первичном этапе.

Рассмотрены основные интерфейсы и способы интеграции LLM в приложения, а также выделены общие точки привязки к поставщику модели: управление параметрами доступа, создание клиента, формирование запроса и контекста, вызов модели и извлечение текста ответа, обработка ошибок и представление результата пользователю. Эти элементы повторяются в большинстве проектов независимо от используемых библиотек и уровня абстракции и, следовательно, являются целевыми объектами для аннотирования и преобразования.

Таким образом, результаты главы 2 уточняют контекст практической части работы и обосновывают выбор направления: разработку решения,

предназначенного для обработки массива проектов и снижения трудоёмкости миграции за счёт формализованных правил аннотирования, воспроизводимого запуска и методики предварительной оценки сложности преобразования. В следующей главе приводится описание разработанного решения, формата правил, процесса аннотирования и результаты апробации подхода на выбранных проектах.

ГЛАВА 3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1. Постановка практической задачи и требования к решению

Практическая часть работы направлена на снижение трудоёмкости первичного этапа миграции LLM-зависимых проектов на альтернативных поставщиков моделей. В типовой ситуации разработчик располагает набором готовых прототипов, реализующих сходные пользовательские сценарии, однако отличающихся используемыми библиотеками, способом обращения к модели, формированием запроса, обработкой ответа и организацией интерфейса. При переносе таких проектов возникает необходимость быстро определить, какие участки исходного кода завязаны на конкретного поставщика модели, и какие изменения потребуются для адаптации приложения.

Основная практическая проблема состоит в том, что точки интеграции LLM распределены по коду и часто создаются в разных формах: импорт и инициализация клиентских библиотек, получение параметров доступа, формирование запроса и структуры сообщений, вызов модели, извлечение текста результата, обработка ошибок, вывод ответа пользователю, а также подключение внешних инструментов. Ручной поиск таких мест в каждом проекте по отдельности приводит к повторяемым действиям, увеличивает риск пропуска значимых фрагментов и усложняет сравнение проектов между собой.

В рамках практической части формулируется задача разработки решения, которое позволяет автоматизировать выделение и фиксацию типовых точек интеграции LLM в исходном коде и тем самым упростить дальнейшую ручную адаптацию проекта под другого поставщика модели. Для этого используется подход аннотирования: к найденным фрагментам кода добавляются комментарии с идентификаторами применённых правил и кратким пояснением. Итогом работы должно стать получение преобразованной версии файла, содержащей аннотации, а также отчёта о выполненных срабатываниях правил и их описания.

Дополнительно применяется методика оценки сложности преобразования проекта. В работе она используется как вспомогательный механизм, позволяющий уточнять классификацию проектов по переносимости и обосновывать различия между типовыми случаями. При этом основное значение сохраняют выявленные точки интеграции и фактические ограничения, а не формальная сумма баллов.

С учётом поставленной задачи к разрабатываемому решению предъявляются следующие требования:

а) автоматическое выделение точек интеграции LLM: решение должно находить в исходном коде ключевые фрагменты, связанные с подключением модели, параметрами доступа, формированием запросов, вызовами модели и обработкой результата.

б) формализуемость и расширяемость: правила аннотирования должны храниться отдельно от кода утилиты и допускать расширение без переработки основной логики работы инструмента.

в) прозрачность результата: преобразованный код должен оставаться читаемым и пригодным для ручной доработки. Аннотации должны указывать идентификатор правила и давать понятную подсказку о назначении отмеченного места.

г) воспроизводимость применения: процесс запуска инструмента и получение результатов должны быть повторяемыми при работе с разными проектами и на разных машинах.

д) Отчётность: помимо преобразованного файла должно формироваться текстовое описание результатов: какие правила применялись, сколько раз и к каким участкам кода они были применены. Отчёт должен облегчать проверку и сравнение проектов, а также фиксировать выполненные действия.

Таким образом, в практической части работы разрабатывается инструмент и сопутствующие материалы, обеспечивающие аннотирование LLM-зависимого кода по набору правил и формирование отчёта. Это создаёт основу для

упрощённого анализа проектов и подготовки к их миграции на отечественные платформы больших языковых моделей.

3.2. Общая архитектура разработанного решения

Разработанное решение представляет собой набор взаимосвязанных компонентов, предназначенных для обработки исходного кода LLM-зависимых проектов и получения результатов, удобных для дальнейшей миграции на альтернативных поставщиков моделей. Архитектура ориентирована на работу с отдельными файлами и проектами, сокращающей объём ручной работы на этапе анализа и первичной подготовки проекта.

В общем виде решение включает три функциональных уровня: описание правил, исполнение преобразования и получение результатов.

а) уровень правил:

На уровне правил задаётся формализованное описание того, какие фрагменты кода необходимо обнаруживать и каким образом фиксировать их в результатах. Правила представлены в виде отдельного набора, который хранится вне кода утилиты и может расширяться по мере накопления новых наблюдений. Это обеспечивает прозрачность преобразований, поскольку итог зависит от конкретной версии набора правил, применённой к исходному файлу.

Каждое правило ориентировано на типовую точку интеграции LLM: подключение библиотек, ввод параметров доступа, создание клиента, вызов модели, извлечение текста ответа, проверки конфигурации и обработку ошибок. В практической части правила используются для аннотирования: в найденные места вставляются пометки с идентификатором правила и предложенным фрагментом кода. Более развёрнутые пояснения по каждому правилу и сведения о срабатываниях фиксируются в отдельном отчёте.

б) уровень обработки и аннотирования:

Центральным компонентом является утилита аннотирования, которая получает на вход исходный файл и набор правил. На этом этапе выполняются следующие действия:

- чтение исходного кода и загрузка набора правил;
- поиск совпадений фрагментов кода с шаблонами правил;
- вставка аннотаций в найденные участки;
- сохранение преобразованной версии кода.

Результатом обработки становится файл, в котором присутствуют пометки, указывающие на точки интеграции с LLM и потенциальные места, требующие преобразования. Такой результат предназначен для дальнейшей ручной доработки, а также для повышения понимания того, как аналогичная функциональность реализуется при использовании разных библиотек и поставщиков моделей.

в) Уровень результатов и отчётности:

Помимо преобразованного файла формируется отчёт о работе инструмента. Отчёт фиксирует, какие правила применялись, сколько раз они сработали и в каком контексте использовались. Это позволяет:

- быстро оценить насыщенность проекта точками интеграции LLM;
- сравнивать проекты между собой по числу и типу найденных мест;
- документировать процесс преобразования и результаты применения правил.

Таким образом, выходными результатами решения являются:

- преобразованный файл с аннотациями, позволяющими автоматически найти большинство мест, требующих внимания при смене поставщика модели, и при необходимости заменить обнаруженные фрагменты предложенным кодом;
- отчёт, отражающий применение правил и служащий основой для анализа проекта.

г) Обеспечение воспроизводимости запуска:

Для уменьшения зависимости от локального окружения и упрощения повторного применения решения используется подход воспроизводимого запуска. Он предполагает наличие фиксированной среды выполнения для утилит и демонстрационных прототипов, что снижает влияние расхождений в версиях библиотек и настройках системы. В рамках архитектуры данный аспект

рассматривается как вспомогательный, обеспечивающий повторяемость экспериментов и удобство проверки полученных результатов.

В итоге архитектура разработанного решения может быть представлена как последовательность преобразования:

исходный код проекта + набор правил → утилита аннотирования → преобразованный код + отчёт.

Представленная структура обеспечивает разделение ответственности между правилами и механизмом их применения, даёт возможность расширять набор правил без изменения основной логики инструмента и обеспечивает прозрачность результата для последующей ручной миграции проекта.

3.3. Формат правил аннотирования и принцип их применения

Ключевым элементом разработанного решения является набор правил аннотирования, который задаёт, какие фрагменты кода необходимо обнаруживать и какие пометки следует вставлять в результирующий файл. Правила вынесены в отдельный файл `rules.json`, что позволяет изменять и расширять набор без переработки логики аннотатора.

Каждое правило в наборе представляет собой объект, содержащий следующие поля:

а) `id` — уникальный идентификатор правила. Он вставляется в код как метка `#id ...` и используется для фиксации срабатываний в отчёте.

б) `pattern_type` — тип или категория правила (`imports`, `key_check`). Поле используется для дополнительной классификации в отчёте и при анализе набора проектов.

в) `comment` — развёрнутое текстовое пояснение к правилу. Оно не вставляется в код, а выводится в `report.txt`, чтобы не перегружать преобразованный файл.

г) `find` — строковый шаблон, по которому выполняется поиск в исходном коде. Поиск основан не на простом сравнении строк, а на сравнении последовательностей токенов, что снижает зависимость от форматирования и

переносов строк. В шаблоне поддерживаются плейсхолдеры вида \$NAME. Плейсхолдер соответствует любому идентификатору языка Python (например имени переменной или объекта), что позволяет описывать типовые конструкции без жёсткой привязки к конкретным именам в проекте.

д) `insert.where` — положение вставки. Поддерживаются два режима: `start` — вставка выполняется в начале файла (после служебных строк вида `shebang` и указания кодировки при их наличии), и `after` — вставка выполняется после строки, на которой заканчивается найденное совпадение.

е) `insert.text` — текст, который будет добавлен в преобразованный файл в виде закомментированного блока.

Для повышения устойчивости к различиям оформления поиск выполняется по токенизированному представлению кода. При токенизации исключаются элементы, которые не влияют на смысл конструкции и могут различаться между проектами: пустые строки, отступы, комментарии и служебные маркеры. Это позволяет распознавать одинаковые фрагменты даже при различиях в переносах строк и стиле форматирования.

Сопоставление выполняется скользящим окном: шаблон сравнивается с последовательными фрагментами токенов исходника. Для литеральных элементов требуется полное совпадение типа и значения токена. Для плейсхолдеров допускается совпадение с любым идентификатором, что делает правила менее чувствительными к конкретным именам переменных.

Результатом этапа сопоставления является список найденных совпадений, для каждого из которых фиксируется диапазон строк исходного файла. Именно этот диапазон используется далее для вычисления позиции вставки при режиме `after`.

Аннотация, добавляемая в файл, имеет компактный формат, чтобы не перегружать исходный код:

- строка с идентификатором правила: `#id ...`;
- далее — предложенные строки кода, записанные в виде комментариев.

Такое представление решает две задачи. Во-первых, оно позволяет быстро находить места, относящиеся к миграции, по идентификаторам и по характерным блокам комментариев. Во-вторых, оно не изменяет исполняемую логику файла, поскольку вставляемые строки остаются комментариями и не влияют на выполнение программы до тех пор, пока разработчик не выполнит целевую замену вручную.

Развёрнутая информация о назначении правила и условиях применения переносится в отчёт `report.txt`. В результате преобразованный файл сохраняет читаемость, а отчёт выполняет роль документации по применённым правилам и местам их срабатывания.

С практической точки зрения набор правил отражает накопленные знания о типовых точках интеграции LLM. Каждое правило фиксирует конкретный узел привязки проекта к провайдеру модели. После аннотирования разработчик получает файл, в котором большинство таких мест выделены и снабжены подсказками, что существенно уменьшает объём ручного поиска и ускоряет первичную подготовку проекта к миграции на альтернативные платформы.

Полные примеры файла правил и отчёта приведены в приложениях. ПРИЛОЖЕНИЕ 3 содержит пример хранителя правил и отчёта, а исходный и преобразованный вариант демонстрационного файла — в ПРИЛОЖЕНИЕ 1.

3.4. Алгоритм работы аннотатора и формирование результатов

Работа аннотатора организована как последовательность действий, выполняемых над входным файлом и набором правил. На вход подаются: исходный Python-файл и `rules.json`. На выходе формируются: преобразованный файл с аннотациями и отчёт `report.txt`.

алгоритм обработки можно представить следующими этапами:

а) загрузка входных данных: аннотатор читает исходный файл целиком и загружает список правил из `rules.json`. Набор правил рассматривается как внешний артефакт: изменение правил не требует модификации самой программы.

б) подготовка представления исходного кода: для поиска совпадений исходный код переводится в последовательность токенов языка Python. При этом исключаются элементы, не влияющие на смысл конструкций и сильно зависящие от оформления: переносы строк, отступы, комментарии и служебные маркеры. Такое представление позволяет находить один и тот же фрагмент независимо от форматирования.

в) подготовка шаблонов поиска из правил: для каждого правила берётся шаблон `find` и приводится к токенизированному виду. Если в шаблоне присутствуют плейсхолдеры вида `$NAME`, они интерпретируются как “произвольный идентификатор” и позволяют сопоставлять конструкции, где имена переменных или объектов отличаются от проекта к проекту.

г) поиск совпадений и определение диапазонов строк: шаблон сравнивается с последовательными фрагментами токенов исходника (скользящее окно). При совпадении фиксируется диапазон строк исходного файла, на котором расположен найденный фрагмент. Эти границы используются для выбора позиции вставки в режиме `after`.

д) формирование операций вставки: для каждого срабатывания правила создаётся операция вставки: куда вставлять и какой блок строк вставить. Поддерживаются два режима: `start` и `after`. Вставляемый блок состоит из строки `#id ...` и последующих строк подсказки, оформленных как комментарии. Отступ вставки подбирается по отступу строки, рядом с которой выполняется вставка, чтобы аннотации визуально совпадали с уровнем кода.

е) применение вставок к тексту: все вставки применяются к исходному тексту. Чтобы позиции не “съезжали” из-за изменения количества строк, вставки выполняются начиная с нижних участков файла к верхним. Это обеспечивает корректность индексов строк и устойчивость результата при множестве вставок.

ж) формирование отчёта `report.txt`: параллельно с подготовкой вставок собираются сведения о сработавших правилах: идентификатор, тип, пояснение, количество срабатываний и места в файле. Отчёт формируется только по правилам, которые реально применились, чтобы не перегружать документ. В

результате report.txt выполняет роль сводной документации: по нему можно быстро оценить насыщенность файла точками интеграции LLM и сопоставлять проекты между собой по набору сработавших правил.

Итогом работы аннотатора являются два взаимодополняющих результата: преобразованный файл, где места потенциальных изменений отмечены непосредственно в коде, и отчёт, содержащий развёрнутую сводку по применённым правилам. Полные листинги входного и выходного файла приведены в Приложении 1, а примеры набора правил и отчёта — в Приложении 3.

3.5. Набор правил и покрываемые точки интеграции LLM

Сформированный набор правил аннотирования ориентирован на выявление типовых точек привязки LLM-зависимого приложения к конкретному провайдеру и его библиотеке. В качестве таких точек рассматриваются участки кода, которые при миграции чаще всего требуют изменения или проверки корректности: подключение SDK, ввод параметров доступа, инициализация клиента, вызов модели и извлечение ответа.

Покрываемые набором правил группы точек интеграции можно описать следующим образом:

а) подключение библиотек провайдера (imports):

Правила этого класса выявляют места импорта библиотек, через которые проект связывается с конкретным API. В результате в начало файла может добавляться блок-подсказка с альтернативными импортами для целевых платформ;

б) получение и хранение параметров доступа (key input): отдельная группа правил ориентирована на места, где вводятся или подготавливаются ключи доступа и идентификаторы. В прототипах это часто реализовано через UI, но общая идея одинакова: в коде присутствует участок, который отвечает за наличие параметров доступа и их дальнейшее использование;

в) проверка конфигурации перед запуском запроса (key check): до обращения к модели в проектах обычно присутствует проверка наличия ключа

или иных обязательных параметров. При смене провайдера меняются требуемые поля и логика проверки, поэтому фиксация таких мест важна для миграции. В результат вставляется подсказка, отражающая альтернативный вариант проверки;

г) инициализация клиента или объекта модели (client init): правила этой группы отмечают места создания клиента, где используются параметры доступа и настройки. Этот участок является одной из центральных точек привязки к SDK, поэтому его выделение упрощает дальнейшую ручную замену;

д) вызов модели и извлечение результата (call / response extract): Даже при одинаковом смысловом шаге вызвать модель и получить текст библиотеки отличаются формой ответа и способом извлечения данных. Поэтому правила фиксируют участки, где производится получение текста результата (например, доступ к вложенным полям объекта ответа). Вставляемые подсказки показывают, что именно в этих местах потребуется адаптация под другой интерфейс.

3.6. Обеспечение воспроизводимости запуска

При работе с массивом LLM-зависимых прототипов важно обеспечить возможность повторного запуска в одинаковых условиях. На практике воспроизводимость часто нарушается из-за различий в версиях интерпретатора Python, наборе установленных библиотек, особенностях операционной системы и локальных настройках окружения. Для проектов, зависящих от внешних библиотек и сетевых сервисов, такие расхождения могут приводить к ошибкам установки, несовместимости зависимостей и отличиям поведения приложений.

В рамках данной работы воспроизводимость обеспечивается двумя взаимодополняющими способами:

а) фиксация зависимостей: для демонстрационного проекта и сопутствующих утилит используется файл requirements.txt, в котором перечислены используемые библиотеки. Это позволяет воспроизвести

окружение на другой машине стандартной установкой зависимостей и избежать ситуации, когда запуск зависит от “случайно установленных” пакетов.

б) контейнеризация с использованием Docker: Docker применяется как средство изоляции и фиксации среды выполнения. Контейнер включает конкретную версию Python и необходимый набор библиотек, что уменьшает влияние различий между системами. В работе используются два варианта контейнеров:

- контейнер для запуска аннотатора: образ содержит минимальную среду выполнения и сам скрипт аннотирования, а входные файлы (исходник и rules.json) передаются через подключение рабочей директории;

- контейнер для запуска преобразованного Streamlit-приложения: образ устанавливает зависимости из requirements.txt, а запуск приложения выполняется через универсальный входной скрипт entrypoint.sh, которому передаётся имя файла. Такой подход позволяет использовать единый образ для проверки разных преобразованных файлов без изменения самого Dockerfile. Для ускорения сборки и исключения лишних артефактов используется файл .dockerignore. Он предотвращает попадание в контекст сборки временных и служебных файлов. Это уменьшает размер образа, ускоряет сборку.

Таким образом, воспроизводимость запуска обеспечивается сочетанием фиксируемых зависимостей и контейнерного окружения: аннотатор и демонстрационные проекты могут быть запущены на разных системах (в том числе в Linux и Windows) с минимальными различиями в подготовке среды. Полные листинги файлов, обеспечивающих контейнеризацию и запуск, приведены в Приложении 4.

3.7. Экспериментальное применение и анализ результатов

Экспериментальная проверка подхода выполнена на трёх LLM-зависимых Streamlit-прототипах, различающихся стилем интеграции и набором библиотек: QuickStart App, Chatbot, Chat with search. Для проектов, ориентированных на GigaChat, использовался набор правил “GigaChat”, для проекта под YandexGPT

— отдельный набор правил “YandexGPT”, чтобы не смешивать несовместимые сценарии миграции.

Таблица 3.1 — Экспериментальные проекты и категория сложности

Проект	Краткое описание	Целевая платформа	Сложность
QuickStart App	Минимальный прототип “запрос–ответ” через LangChain OpenAI	GigaChat	0 баллов = простой
Chatbot	Чат с историей сообщений (session_state), OpenAI SDK	GigaChat	1 балл = простой
Chat with search	Агент с веб-поиском и отображением истории чата	YandexGPT	2 балла = простой

Результаты по проектам:

– QuickStart App (миграция на GigaChat): в проекте обнаружены и отмечены основные узлы интеграции: импорт LangChain OpenAI, ввод ключа, вызов модели в функции генерации ответа, а также проверки формата ключа OpenAI через `startswith('sk-')`. По журналу видно, что часть правил сработала ожидаемо: отдельное правило для импорта LangChain OpenAI, правило для ввода ключа (вариант без `type="password"`) и правила для проверок `startswith`. При этом заметна типичная особенность текущего набора правил: срабатывание правила, связанного с вызовом модели, оказалось возможным только потому, что строка в коде совпала с шаблоном достаточно точно (в том числе по параметру `temperature=0.7`). Это подчёркивает зависимость от жёстких аргументов в шаблонах: при изменении значения (например, 0.66, либо при отсутствии параметра) правило может не сработать, хотя смысловая точка интеграции в проекте присутствует.

– Chatbot (миграция на GigaChat): для данного проекта правила выделили ключевые места: проверку наличия ключа и извлечение ответа из структуры результата OpenAI (`response.choices[0].message.content`), а также добавили подсказку по импортам. Вместе с тем, здесь проявляется ограничение покрытия: правила не фиксируют создание клиента OpenAI(`api_key=...`), если в наборе отсутствует соответствующий шаблон или он не совпал с конкретной записью. Это означает, что при расширении базы преобразований потребуется добавлять

правила, ориентированные на разные варианты инициализации клиента и разные библиотеки.

– Chat with search (миграция на YandexGPT): эксперимент показал, что отдельный набор правил под YandexGPT корректно отмечает четыре ключевых изменения: замену импорта ChatOpenAI, добавление параметров доступа key, изменение проверки наличия параметров и замену инициализации LLM на YandexGPT(...). При этом логика агента, веб-поиска и обработчиков Streamlit не затрагивается, что соответствует цели аннотирования: выделять точки привязки к провайдеру, не вмешиваясь в остальную логику приложения. Ограничение здесь аналогичное: правило инициализации модели сработало только при точном совпадении строки вызова ChatOpenAI(...) с шаблоном; альтернативные записи (другие имена аргументов, иной порядок параметров) могут потребовать отдельных шаблонов.

В результате эксперимента можно зафиксировать несколько практических ограничений, которые следует учитывать при дальнейшем расширении решения:

а) зависимость от точного совпадения аргументов: часть правил формулируется как достаточно “жёсткий” шаблон. Например, правило может совпасть только при наличии конкретного набора аргументов (temperature=0.7) или при конкретной записи вызова (в одну строку, с определёнными именами параметров). Это приводит к тому, что при небольших изменениях значения или формы записи правило не срабатывает, хотя смысловая точка интеграции присутствует;

б) неполное покрытие вариантов интеграции: даже в рамках трёх проектов видно, что разные библиотеки и стили кода требуют отдельных правил. Например, OpenAI может подключаться как через OpenAI из SDK, так и через OpenAI в LangChain; для ввода ключа встречаются разные варианты text_input; проверка может быть if not key, а может быть startswith('sk-'). Для расширения базы преобразований требуется наращивание набора шаблонов под распространённые варианты.

в) результат в основном зависит от качества базы правил, а не только универсальности механизма: механизм аннотирования остаётся одинаковым, но практическая полезность напрямую определяется тем, насколько полно набор правил покрывает типовые реализации. Это соответствует выбранной концепции: решение не угадывает намерения разработчика, а применяет накопленные и формализованные знания в виде правил.

Вывод по эксперименту:

Эксперимент подтверждает, что подход работоспособен в качестве инструмента первичного анализа и подготовки к миграции: в трёх прототипах он выделил основные точки интеграции, относящиеся к подключению провайдера, параметрам доступа, инициализации модели и получению ответа. Одновременно эксперимент выявил ключевое направление развития решения — расширение и смягчение базы правил, чтобы уменьшить зависимость от точных значений аргументов и повысить устойчивость к различным стилям записи кода.

3.8. Выводы по главе 3

В третьей главе разработано и описано практическое решение, направленное на снижение трудоёмкости первичного анализа LLM-зависимых проектов при подготовке их миграции на отечественные платформы. Определены состав и роль основных компонентов: внешний набор правил, утилита аннотирования и формирование результатов в виде размеченного файла и отчёта о срабатываниях.

Показано, что правила позволяют фиксировать типовые точки интеграции LLM: подключение библиотек провайдера, ввод и проверку параметров доступа, инициализацию клиента/модели, вызовы модели и извлечение ответа. За счёт вынесения правил в отдельный JSON-файл обеспечивается расширяемость базы преобразований без изменения кода аннотатора, а также воспроизводимость результата при повторном применении одного и того же набора правил.

Описан алгоритм работы аннотатора, основанный на токенизированном представлении исходного кода, что повышает устойчивость к различиям

форматирования по сравнению с простым строковым поиском. Рассмотрены меры по обеспечению воспроизводимости запуска: фиксация зависимостей и контейнеризация Docker.

Экспериментальная апробация на трёх проектах подтвердила применимость подхода для типовых прототипов и позволила выявить его ограничения. Основные недостатки связаны с зависимостью правил от конкретной формы записи конструкций и значений аргументов, а также с неполным покрытием вариантов интеграции. Это определяет направление дальнейшего развития — расширение набора правил и повышение их устойчивости к различным стилям кода при сохранении прозрачности и контролируемости аннотирования.

Таким образом, результаты главы 3 демонстрируют, что предложенное решение может служить практической основой для системной работы с массивом LLM-зависимых проектов: оно ускоряет поиск ключевых мест в коде, структурирует процесс подготовки к миграции и формирует воспроизводимый набор артефактов для дальнейшей ручной адаптации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была решена задача снижения трудоёмкости первичного этапа миграции LLM-зависимых Streamlit-приложений на Python между различными поставщиками моделей за счёт формализованного аннотирования исходного кода по набору правил и применения методики предварительной оценки сложности преобразования, позволяющей заранее определить ключевые места, требующие внимания при переносе.

Для достижения цели проанализированы типовые паттерны интеграции LLM в Streamlit-проекты, рассмотрены существующие подходы к унификации доступа к моделям и анализу/преобразованию кода, сформированы требования к решению (прозрачность, расширяемость, воспроизводимость, отчётность). Разработаны формат правил и механизм их применения, предусматривающий формирование преобразованного файла и текстового отчёта, а также предложена классификация проектов по переносимости на основе оценки сложности.

Практическим результатом работы стало решение, состоящее из четырёх ключевых факторов:

- формализацию аспектов, на которые необходимо обращать внимание при миграции LLM-зависимого приложения;
- набор правил, хранящийся отдельно от кода утилиты и расширяемый по мере накопления новых паттернов;
- утилита аннотирования, выполняющая поиск совпадений по правилам и вставку пометок в исходный код;
- результирующие артефакты в виде преобразованного файла и текстового отчёта, фиксирующего какие правила применялись, сколько раз и к каким участкам кода. Такой подход обеспечивает прозрачность преобразований и делает процедуру анализа воспроизводимой.

Для повышения устойчивости к различиям форматирования предложен алгоритм аннотирования, основанный на поиске в токенизированном

представлении исходного кода, что в типовых случаях надёжнее простого строкового поиска. Воспроизводимость запуска обеспечена фиксацией зависимостей (requirements.txt) и контейнеризацией Docker как для утилиты аннотирования, так и для демонстрационных Streamlit-прототипов.

Апробация выполнена на трёх LLM-зависимых Streamlit-проектах (QuickStart App, Chatbot, Chat with search) с применением отдельных наборов правил для GigaChat и YandexGPT. Результаты показали, что для типовых прототипов подход позволяет выделять основные точки интеграции и формировать отчёт, упрощающий дальнейшую ручную адаптацию.

Ограничения решения связаны с зависимостью части правил от конкретной формы записи конструкций и аргументов, а также с ограниченным покрытием вариантов интеграции, что требует расширения базы шаблонов. Перспективы развития — увеличение числа поддерживаемых паттернов и переход к более структурным методам анализа (AST/CST) для повышения устойчивости преобразований при сохранении их контролируемости.

Таким образом, полученные результаты позволяют сделать вывод, что разработанный подход может служить практической основой для системной работы с массивом LLM-зависимых проектов: он ускоряет поиск ключевых мест в коде, стандартизирует первичную подготовку к миграции и формирует воспроизводимый набор артефактов, упрощающий дальнейшую ручную адаптацию. Перспективы развития решения связаны с расширением и смягчением базы правил, а также с возможным переходом к более структурным методам анализа для повышения устойчивости при сохранении контролируемости преобразований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Намиот Д.Е., Ильюшин Е.А. Архитектура LLM агентов /International Journal of Open Information Technologies. 2025. Т. 13. № 1. с. 67–74.
2. Brown T.B., Mann B., Ryder N. и др. Language Models are Few-Shot Learners / Advances in Neural Information Processing Systems (NeurIPS). 2020. Режим доступа: <https://arxiv.org/abs/2005.14165>
3. OpenAI. API Reference. — Режим доступа: <https://platform.openai.com/>
4. Сбер. GigaChat API: Overview. — Режим доступа: <https://developers.sber.ru/>
5. Yandex Cloud. YandexGPT. — Режим доступа: <https://yandex.cloud/>
6. Streamlit. Build a basic LLM chat app. — Режим доступа: <https://docs.streamlit.io/>
7. Docker. Get started. — Режим доступа: <https://docs.docker.com/>
8. Python Software Foundation. tokenize — Tokenizer for Python source. — Режим доступа: <https://docs.python.org/>
9. Python Software Foundation. What's New In Python 3.13— Режим доступа: <https://docs.python.org/3/whatsnew/3.13.html>
10. Bowler. Refactoring basics (fixers, паттерны и трансформации). — Режим доступа: <https://pybowler.io/>
11. Чуркин Я.А., Мельник Д.М., Бучацкий Р.А. Обзор методов миграции программных интерфейсов приложений для объектно-ориентированных языков / Труды ИСП РАН. 2025. Т. 37. Вып. 4. Ч. 1. с. 111–146.
12. Алпатов А.Н., Соловьев Р.В. Архитектура программной системы автоматизации рефакторинга кода на основе метода комбинаторных парсеров / Современные наукоемкие технологии. 2022. № 4. с. 9–15
13. Антонов И. В., Бруттан Ю. В. Применение больших языковых моделей для интеллектуального поиска и извлечения информации в корпоративных информационных системах / Компьютерные исследования и моделирование. 2025. Т. 17, № 5. с. 871–888.

ПРИЛОЖЕНИЕ 1. ПРИМЕР ПРОГРАММЫ ДО И ПОСЛЕ ИЗМЕНЕНИЯ

В приложении приведён демонстрационный пример LLM-зависимого Streamlit-приложения в двух вариантах: исходный файл и файл после аннотирования.

Листинг 1.1 — ProgramForConversion.py (исходный файл):

```
# Основные точки интеграции LLM в этом файле:
# 1) Импорт библиотеки провайдера модели (OpenAI).
# 2) Ввод и хранение ключа доступа.
# 3) Проверка наличия ключа перед обращением к модели.
# 4) Создание клиента и вызов модели.
# 5) Извлечение текста ответа из объекта результата.
# 6) Вывод сообщений и хранение истории диалога.
from openai import OpenAI
import streamlit as st

# Блок боковой панели: пользователь вводит ключ доступа.
# В прототипах это часто делается через UI, чтобы не хранить ключи в коде.
with st.sidebar:
    openai_api_key = st.text_input(
        "OpenAI API Key",
        key="chatbot_api_key",
        type="password"
    )

# Заголовок и краткое описание приложения.
st.title("🗨️ Chatbot")
st.caption("🔗 A Streamlit chatbot powered by OpenAI")

# Хранение истории сообщений в st.session_state:
# это обеспечивает "память" диалога в пределах одной пользовательской сессии.
# Если история ещё не создана — добавляем стартовое сообщение ассистента.
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "assistant", "content": "How can I help you?"}
    ]

# Отрисовка всей истории диалога.
# Каждый элемент истории — словарь вида {"role": "...", "content": "... "}.
for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"])

# Получение нового пользовательского сообщения.
# st.chat_input возвращает строку, если пользователь отправил сообщение.
if prompt := st.chat_input():
    # Проверка, что ключ введён. Без ключа запрос к модели невозможен.
    if not openai_api_key:
        st.info("Please add your OpenAI API key to continue.")
        st.stop()

    # Создание клиента провайдера модели.
    # Важно: способ создания клиента и параметры инициализации
    # являются точкой привязки к конкретному поставщику LLM.
    client = OpenAI(api_key=openai_api_key)

    # Добавляем сообщение пользователя в историю и выводим его в интерфейсе.
```

```

st.session_state.messages.append({"role": "user", "content": prompt})
st.chat_message("user").write(prompt)
# Вызов модели: передаём историю сообщений целиком.
# В разных библиотеках может отличаться:
# - имя метода,
# - формат входных сообщений,
# - структура объекта ответа.
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=st.session_state.messages
)
# Извлечение текста ответа из результата.
# Это ещё одна типовая точка, которая меняется при смене SDK.
msg = response.choices[0].message.content
# Добавляем ответ ассистента в историю и выводим пользователю.
st.session_state.messages.append({"role": "assistant", "content": msg})
st.chat_message("assistant").write(msg)

```

Листинг 1.2 — output.py (результат аннотирования)

```

# Важно: аннотации содержат:
# - идентификатор правила (например, "#id 001"),
# - предложенный фрагмент кода виде закомментированных строк.
# Подробные пояснения (что означает правило, зачем оно нужно, что именно менять)
# фиксируются в отчёте преобразования, а в коде остаются только краткие пометки.
#id 001
# Возможная альтернатива: подключение GigaChat через совместимую библиотеку.
# В данном файле строки оставлены закомментированными как подсказка при миграции.
# from langchain_community.chat_models.gigachat import GigaChat
# from langchain.schema import HumanMessage
from openai import OpenAI
import streamlit as st
with st.sidebar:
    openai_api_key = st.text_input("OpenAI API Key", key="chatbot_api_key", type="password")
st.title("🤖 Chatbot")
st.caption("📄 A Streamlit chatbot powered by OpenAI")
if "messages" not in st.session_state:
    st.session_state["messages"] = [{"role": "assistant", "content": "How can I help you?"}]
for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"])
if prompt := st.chat_input():
    if not openai_api_key:
        #id 005
        # Пример альтернативной проверки параметров доступа для другого провайдера.
        # Если проект мигрируется на GigaChat, в реальном коде проверка будет иной.
        # if not gigaChatKey:
        st.info("Please add your OpenAI API key to continue.")
        st.stop()
        # Создание клиента OpenAI (в исходной версии остаётся без изменения).
        # При миграции на другого провайдера этот блок является одним из ключевых
        # кандидатов на замену.
        client = OpenAI(api_key=openai_api_key)
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)

```



```
response = client.chat.completions.create(model="gpt-3.5-turbo", messages=st.session_state.messages)
# Извлечение текста ответа из объекта результата.
msg = response.choices[0].message.content
#id 004
# Пример альтернативного получения ответа (схема для другого SDK):
# message = [HumanMessage(content=prompt)]
# response = (client(message)).content
# Примечание: строки приведены как заготовка, но в реальном переносе
# требуется согласовать формат сообщений и способ вызова модели.
st.session_state.messages.append({"role": "assistant", "content": msg})
st.chat_message("assistant").write(msg)
```

ПРИЛОЖЕНИЕ 2. ПРОГРАММА АННОТАТОР

В данном приложении приведён полный исходный код утилиты аннотирования.

Листинг 2.1 — Annotator.py (утилита аннотирования)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import argparse # удобный разбор аргументов командной строки
import io       # io — используем StringIO, чтобы токенизатор tokenize мог читать код как файл
import json     # json — чтение набора правил из rules.json
import re       # re — регулярные выражения используются для плейсхолдеров
import tokenize # tokenize — разбор Python-кода на токены (имена, скобки, строки, операторы и тд)
from dataclasses import dataclass # dataclass — сокращает написание классов-структур данных
from datetime import datetime     # datetime — для отметки времени в отчёте report.txt
from typing import Any, Dict, List, Optional # typing - аннотация типов для читаемости и самопроверки
# Константы, связанные с плейсхолдерами в шаблонах поиска
PH_PREFIX = "__PH__" # Префикс для "служебного" имени плейсхолдера
PH_SUFFIX = "_"     # Суффикс для "служебного" имени плейсхолдера
# Регулярное выражение для плейсхолдеров в find-шаблоне:
# $NAME, $VAR1, $some_id. То есть: знак $ + идентификатор
PLACEHOLDER_RE = re.compile(r"\$[A-Za-z_][A-Za-z0-9_]*")
# Структуры данных для скомпилированного шаблона и найденных совпадений
@dataclass # Декоратор dataclass автоматически генерирует __init__, __repr__, __eq__ и др.
class PatternTok: # PatternTok — элемент шаблона, с которым мы будем сравнивать токены исходника
    # kind определяет тип элемента шаблона:
    # - "LIT" (literal) — точное совпадение токена по типу и строке (например, NAME == "OpenAI")
    # - "PH" (placeholder) — плейсхолдер: соответствует "любому имени" (tokenize.NAME), например $X
    kind: str # "LIT" или "PH"
    # Для LIT-элемента:
    ttype: Optional[int] = None # тип токена (прим: tokenize.NAME)
    tstr: Optional[str] = None  # строковое значение токена (прим: ".")
    # Для PH-элемента:
    ph_name: Optional[str] = None # имя плейсхолдера без "$" (прим: $X -> "X")
@dataclass
class MatchSpan: # MatchSpan описывает, на каких строках исходника найдено совпадение
    # Важно: строки считаются в стиле "1-based", то есть первая строка = 1
    start_line: int # номер стартовой строки диапазона совпадения
    end_line: int  # номер конечной строки диапазона совпадения

# Игнорируемые токены при токенизации исходного кода
# Требуется сравнивать "смысловые" токены кода (имена, операторы, строки, числа),
# Нужно чтобы совпадения не ломались из-за форматирования, пустых строк и комментариев.
IGNORE_TOKEN_TYPES = {
    tokenize.ENCODING, # Токен кодировки файла
    tokenize.NL,       # Неформатный перенос внутри выражений
    tokenize.NEWLINE,  # Конец логической строки
    tokenize.INDENT,   # Увеличение отступа (начало блока после if/for/def/class)
    tokenize.DEDENT,   # Уменьшение отступа (выход из блока)
    tokenize.COMMENT,  # Комментарии (# ...)
```

```

    tokenize.ENDMARKER, # Конец потока токенов (служебный маркер завершения)
}
# Вспомогательные функции ввода/вывода текста
def read_text(path: str) -> str:
    # Читает текстовый файл целиком в UTF-8.
    # Используется для чтения исходного .py файла.
    with open(path, "r", encoding="utf-8") as f:
        return f.read()
def write_text(path: str, text: str) -> None:
    # Записывает текстовый файл целиком в UTF-8.
    # Используется для сохранения output.py и report.txt.
    with open(path, "w", encoding="utf-8") as f:
        f.write(text)
def leading_ws(s: str) -> str:
    # (пробелы/табуляции)
    # Возвращает отступ в начале строки чтобы вставленные комментарии имели тот же отступ, что и
    код рядом.
    return s[: len(s) - len(s.lstrip(" \t"))]
# Токенизация Python-кода и подготовка шаблонов поиска
def tokenize_python(code: str) -> List[tokenize.TokenInfo]:
    # Токенизация Python-кода с отбрасыванием "шума":
    # Цель: чтобы один и тот же код, написанный с разными переносами/отступами,
    # давал одинаковую последовательность значимых токенов для поиска совпадений.
    out: List[tokenize.TokenInfo] = []
    # tokenize.generate_tokens ждёт функцию readline, как у файла.
    # Поэтому мы превращаем строку code во файлоподобный объект.
    reader = io.StringIO(code).readline
    # Генерируем токены по всему коду
    for tok in tokenize.generate_tokens(reader):
        # Если токен относится к "шуму" — пропускаем
        if tok.type in IGNORE_TOKEN_TYPES:
            continue
        # Иначе — сохраняем
        out.append(tok)
    return out
def preprocess_placeholders(pattern: str) -> str:
    # В правилах поиска допускаются плейсхолдеры вида $NAME.
    # Проблема: "$NAME" не является валидным идентификатором Python-кода.
    # Решение: перед токенизацией мы заменяем "$NAME" на "__PH_NAME__".
    # Далее при компиляции шаблона токен "__PH_NAME__" будет распознан как placeholder ("PH").
    def repl(m: re.Match) -> str:
        # m.group(0) — это совпавшая строка вида "$NAME"
        name = m.group(0)[1:] # убираем '$'
        return f"{PH_PREFIX}{name}{PH_SUFFIX}" # превращаем в "__PH_NAME__"
    # Заменяем все $NAME в строке pattern
    return PLACEHOLDER_RE.sub(repl, pattern)
def compile_pattern(find_text: str) -> List[PatternTok]:
    # Компилирует find-шаблон из rules.json в список PatternTok:
    # - плейсхолдеры ($X) превращаются в элементы kind="PH"
    # - остальные токены становятся kind="LIT" с точным совпадением по type и string
    pre = preprocess_placeholders(find_text) # заменили $NAME -> __PH_NAME__
    toks = tokenize_python(pre) # токенизировали как Python-код
    pat: List[PatternTok] = []

```

```

for t in toks:
    # Если это NAME-токен и он выглядит как "__PH__..." — это плейсхолдер
    if (
        t.type == tokenize.NAME
        and t.string.startswith(PH_PREFIX)
        and t.string.endswith(PH_SUFFIX)
    ):
        # Из "__PH__NAME__" вытаскиваем "NAME"
        name = t.string[len(PH_PREFIX) : -len(PH_SUFFIX)]
        pat.append(PatternTok(kind="PH", ph_name=name))
    else:
        # Иначе это литерал: тип и строка должны совпасть строго
        pat.append(PatternTok(kind="LIT", ttype=t.type, tstr=t.string))
return pat

# Поиск совпадений шаблона в токенах исходного кода
def find_matches_in_tokens(
    src_tokens: List[tokenize.TokenInfo],
    pattern: List[PatternTok],
) -> List[MatchSpan]:
    # Поиск совпадений шаблона по токенам исходника (скользящее окно).
    # Мы двигаем "окно" длины len(pattern) по src_tokens и сравниваем токен за токеном.
    # Если все элементы совпали — считаем, что нашли совпадение.
    if not pattern:
        # Пустой шаблон бессмысленен: нечего искать
        return []
    matches: List[MatchSpan] = []
    n = len(src_tokens) # количество токенов исходника
    m = len(pattern)    # длина шаблона
    if m > n:
        # Если шаблон длиннее исходника — совпадений быть не может
        return matches
    # Скользящее окно: i — индекс начала окна
    for i in range(0, n - m + 1):
        ok = True
        # Сравнение внутри окна
        for j in range(m):
            pt = pattern[j]    # элемент шаблона
            st = src_tokens[i+j] # соответствующий токен исходника

            if pt.kind == "PH":
                # Плейсхолдер: подходит любое "имя"
                if st.type != tokenize.NAME:
                    ok = False
                    break
            else:
                # Литерал: тип и строка должны совпасть полностью
                if st.type != pt.ttype or st.string != pt.tstr:
                    ok = False
                    break
        if ok:
            # Если все сравнения прошли — определяем диапазон строк, где лежит совпадение.
            # tokenize.TokenInfo имеет координаты:
            # t.start = (line, col) — где токен начинается

```

```

# t.end = (line, col) — где токен заканчивается
window = src_tokens[i : i + m]
# start_line — минимальная строка среди токенов окна
start_line = min(t.start[0] for t in window)
# end_line — максимальная строка среди токенов окна
end_line = max(t.end[0] for t in window)
matches.append(MatchSpan(start_line=start_line, end_line=end_line)) # Составление диапазона
return matches

# Определение позиции вставки "в начало файла"
def detect_start_insertion_line(lines: List[str]) -> int:
    # Если правило говорит "вставить в начало файла" (where == "start"),
    # мы не всегда можем вставлять в строку 0
    # В файле может быть указание кодировки или shebang: "#!/usr/bin/env python3"
    # Поэтому мы вычисляем индекс строки, после которой безопасно вставлять.
    idx = 0
    # 1) Пропускаем shebang (если он есть)
    if idx < len(lines) and lines[idx].startswith("#!"):
        idx += 1
    # 2) Пропускаем encoding-cookie, если он есть, включая разный формат написания
    enc_re = re.compile(r"encoding[:=]\\s*[-\\w.]+")
    if idx < len(lines) and enc_re.search(lines[idx]):
        idx += 1
    # Дополнительный случай: иногда shebang на 1-й строке, а encoding на 2-й
    elif idx == 1 and idx < len(lines) and enc_re.search(lines[idx]):
        idx += 1
    return idx

# Формирование блока комментариев, который будет вставлен в output.py
def make_commented_block_with_id(rule_id: str, text: str, indent: str) -> List[str]:
    # Формирует блок строк-комментариев, который будет вставлен в файл.
    # Структура блока:
    # <indent>#id <rule_id>
    # <indent># <строка 1 из text>
    # <indent># <строка 2 из text>
    # ...
    # indent нужен, чтобы вставка выглядела аккуратно и совпадала по отступам с местом вставки.
    out: List[str] = []
    # Первая строка блока — метка правила
    out.append(f"{indent}#{rule_id}\\n")
    # Каждую строку предложенного текста превращаем в комментарий
    for line in text.splitlines():
        if line.strip() == "":
            # Пустую строку тоже сохраняем как комментарий "#", чтобы блок был читаем
            out.append(f"{indent}#\\n")
        else:
            out.append(f"{indent}# {line}\\n")
    # Если текст был полностью пустой (только #id), добавим хотя бы одну пустую "#"
    if len(out) == 1:
        out.append(f"{indent}#\\n")
    return out

# Описание операции вставки и элемента отчёта
@dataclass
class InsertOp: # InsertOp описывает одну конкретную вставку в файл
    # line_index — индекс строки (0-based), в которую вставлять (перед этой строкой)

```

```

# block_lines — список строк, которые нужно вставить
# seq — порядковый номер (нужен для устойчивого порядка, если вставок много в одно место)
line_index: int
block_lines: List[str]
seq: int
@dataclass
class ReportEntry: # ReportEntry — запись для отчёта (одна вставка = одна запись)
    rule_id: str # идентификатор правила
    pattern_type: str # тип правила шаблона
    comment: str # человеческий комментарий правила
    where: str # — где вставляли: start или after
    start_line: Optional[int] = None # start_line/end_line — диапазон строк совпадения
    end_line: Optional[int] = None
def load_rules(path: str) -> List[Dict[str, Any]]: # Загрузка правил и извлечение шаблона из правила
    # Загружаем rules.json.
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    # Защита от неправильного формата: если не список — это ошибка
    if not isinstance(data, list):
        raise ValueError("Корневой элемент JSON должен быть списком правил")
    return data
def rule_find_text(rule: Dict[str, Any]) -> str:
    # В правилах find может быть строкой или объектом.
    # Поддерживаем оба варианта:
    # - если find — dict, берём поле find["pattern"]
    # - иначе преобразуем find в строку
    f = rule.get("find", "")
    if isinstance(f, dict):
        return str(f.get("pattern", ""))
    return str(f or "")
# Построение списка вставок (ops) и списка записей отчёта (report)
def build_ops_and_report(
    rules: List[Dict[str, Any]],
    src_text: str
) -> (List[InsertOp], List[ReportEntry]):
    # Ключевая функция
    lines = src_text.splitlines(keepends=True) # Разбиваем исходник по строкам (с сохранением '\n')
    src_tokens = tokenize_python(src_text) # Токенизируем исходник для устойчивого поиска по токенам
    ops: List[InsertOp] = [] # Сюда собираем все будущие вставки
    report: List[ReportEntry] = [] # Сюда собираем записи для отчёта
    seq = 0 # Счётчик вставок
    for rule in rules:
        # Читаем основные поля правила
        rule_id = str(rule.get("id", "")).strip() or "UNKNOWN" # если id пустой — ставим заглушку
        pattern_type = str(rule.get("pattern_type", "")).strip() # тип/класс паттерна
        comment = str(rule.get("comment", "")).strip() # человеческий комментарий правила

        # Читаем блок insert (куда и что вставлять)
        insert = rule.get("insert", {}) or {}
        where = str(insert.get("where", "")).strip().lower() # "start" или "after"
        ins_text = str(insert.get("text", "") or "") # текст, который станет комментариями
        # Извлекаем find-шаблон
        ftxt = rule_find_text(rule)

```

```

# Вариант вставки в начало файла
if where == "start":
    idx = detect_start_insertion_line(lines) # вычисляем корректную позицию после shebang/encoding
    block = make_commented_block_with_id(rule_id, ins_text, indent="") # в начале отступ пустой
    ops.append(InsertOp(line_index=idx, block_lines=block, seq=seq))
    report.append(ReportEntry(
        rule_id=rule_id,
        pattern_type=pattern_type,
        comment=comment,
        where="start",
        start_line=None,
        end_line=None
    ))
    seq += 1
    continue

# Вариант вставки после найденного совпадения
if where != "after":
    # Если в поле where пришло что-то другое — правило считаем неподдерживаемым и
    пропускаем
    continue
if not ftxt.strip():
    # Если шаблон find пустой — искать нечего
    continue
# Компилируем шаблон find в последовательность PatternTok
pattern = compile_pattern(ftxt)
# Ищем совпадения этого шаблона по токенам исходного кода
matches = find_matches_in_tokens(src_tokens, pattern)
# Для каждого найденного совпадения формируем вставку
for ms in matches:
    # ms.start_line и ms.end_line — 1-based номера строк
    start_line_idx = ms.start_line - 1 # переводим в 0-based индекс строки для списка lines
    # Определяем отступ строки, на которой начинается совпадение,
    indent = ""
    if 0 <= start_line_idx < len(lines):
        indent = leading_ws(lines[start_line_idx])
    # Вставляем после end_line.
    # line_index в InsertOp используется как позиция вставки "перед строкой с индексом".
    # Поэтому если end_line = 10 (1-based), то вставка "после 10-й" означает line_index = 10.
    insert_at = ms.end_line # after => после end_line
    block = make_commented_block_with_id(rule_id, ins_text, indent=indent)
    ops.append(InsertOp(line_index=insert_at, block_lines=block, seq=seq))
    report.append(ReportEntry(
        rule_id=rule_id,
        pattern_type=pattern_type,
        comment=comment,
        where="after",
        start_line=ms.start_line,
        end_line=ms.end_line
    ))
    seq += 1
return ops, report

# Применение вставок к тексту исходника
def apply_ops(src_text: str, ops: List[InsertOp]) -> str:

```

```

# Вставки применяем снизу вверх, чтобы индексы строк не "съезжали".
# Сортируем вставки по line_index по убыванию и вставляем начиная с самых нижних мест
lines = src_text.splitlines(keepends=True)
# Сортировка:
# (line_index, seq) — чтобы если два блока вставляются в одно место,
# сохранялся стабильный порядок по номеру seq.
ops_sorted = sorted(ops, key=lambda o: (o.line_index, o.seq), reverse=True)
for op in ops_sorted:
    # Защита: line_index приводим в допустимые пределы [0 .. len(lines)]
    idx = max(0, min(op.line_index, len(lines)))
    # Вставка списка строк
    lines[idx:idx] = op.block_lines
return "".join(lines)

# Формирование отчёта report.txt
def write_report_txt(report_entries: List[ReportEntry], report_path: str = "report.txt") -> None:
    # Пишем отчёт только по тем правилам, которые реально сработали
    # группируем по rule_id, чтобы не печатать описание правила много раз.
    grouped: Dict[str, Dict[str, Any]] = {}
    for e in report_entries:
        g = grouped.setdefault(e.rule_id, { # Берём группу по rule_id или создаём новую
            "rule_id": e.rule_id,
            "pattern_type": e.pattern_type,
            "comment": e.comment,
            "count": 0,
            "items": []
        })
        g["count"] += 1 # Увеличиваем количество срабатываний правила
        if e.where == "start": # Добавляем человекочитаемое описание места срабатывания
            g["items"].append("вставка: start (в начало файла)")
        else:
            g["items"].append(
                f"совпадение: строки {e.start_line}-{e.end_line}, "
                f"вставка after (после строки {e.end_line})"
            )
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S") # Время формирования отчёта
    # Собираем отчёт построчно в список lines, затем склеиваем
    lines: List[str] = []
    lines.append("ОТЧЁТ О ВСТАВКАХ (report.txt)\n")
    lines.append(f"Время: {now}\n")
    lines.append(f"Срабатываний (вставок) всего: {len(report_entries)}\n")
    lines.append("\n")
    for rule_id in sorted(grouped.keys()):
        g = grouped[rule_id]
        lines.append(f"ID: {g['rule_id']}\n")
        if g["pattern_type"]:
            lines.append(f"Тип: {g['pattern_type']}\n")
        if g["comment"]:
            lines.append(f"Комментарий правила: {g['comment']}\n")
        lines.append(f"Количество вставок: {g['count']}\n")
        lines.append("Где:\n")
        for item in g["items"]:
            lines.append(f"  {item}\n")
        lines.append("\n")

```



```

# Записываем файл report.txt
write_text(report_path, "".join(lines))
# Точка входа (CLI-интерфейс командной строки)
def main() -> None:
    # argparse.ArgumentParser создаёт интерфейс запуска из терминала:
    # python Annotator.py rules.json input.py output.py
    ap = argparse.ArgumentParser(
        description="Прототип: вставка комментариев в Python-код по JSON-правилам и токен-шаблонам."
    )
    # Три обязательных позиционных аргумента:
    ap.add_argument("rules_json", help="Путь к JSON с правилами (новый формат)")
    ap.add_argument("input_file", help="Путь к входному .py файлу")
    ap.add_argument("output_file", help="Путь к выходному .py файлу")
    args = ap.parse_args() # Парсим аргументы командной строки
    # Загружаем правила и читаем исходный файл
    rules = load_rules(args.rules_json)
    src_text = read_text(args.input_file)
    ops, report_entries = build_ops_and_report(rules, src_text) # Строим список вставок и отчётные записи
    out_text = apply_ops(src_text, ops) # Применяем вставки к тексту
    # Сохраняем преобразованный файл и отчёт
    write_text(args.output_file, out_text)
    write_report_txt(report_entries, report_path="report.txt")
    # Сообщение в консоль
    print(
        f"Готово. Вставлено блоков: {len(ops)}. "
        f"Результат: '{args.output_file}'. Отчёт: 'report.txt'."
    )
# Стандартная конструкция Python:
# код внутри if выполнится только если файл запущен как программа,
# а не импортирован как модуль.
if __name__ == "__main__":
    main()

```

ПРИЛОЖЕНИЕ 3. ПРИМЕР ПРАВИЛ АННОТИРОВАНИЯ И ОТЧЁТА

В данном приложении приведены два ключевых артефакта разработанного решения:

`rules.json` — формализованный набор правил аннотирования, который определяет что искать в исходном коде и какой блок пометки вставлять.

`report.txt` — отчёт о применении правил, формируемый аннотатором по результатам обработки конкретного файла.

Листинг 3.1 — `rules.json`:

`id` — уникальный идентификатор правила, используется в пометках;

`pattern_type` — классификация правила используется для группировки/аналитики в отчёте;

`comment` — развёрнутое пояснение назначения правила, используется в `report.txt`;

`find` — строка-шаблон, которая ищется в источнике;

`insert` — описание вставки;

`insert.where` — место вставки, либо в начале файла, либо после строки-совпадения;

`insert.text` — текст, который будет добавлен в файл.

```
[
  {
    "id": "001",
    "pattern_type": "imports",
    "comment": "Подсказка по альтернативным импортам для GigaChat (как ориентир для миграции).",
    "find": {
      "pattern": "from openai import OpenAI"
    },
    "insert": {
      "where": "start",
      "text": "from langchain_community.chat_models.gigachat import GigaChat\nfrom langchain.schema\nimport HumanMessage"
    }
  },
  {
    "id": "002",
    "pattern_type": "key_input",
    "comment": "Пример добавления полей для ввода ключей/идентификаторов альтернативного\nпровайдера.",
    "find": {
      "pattern": "openai_api_key = st.text_input("
    },
    "insert": {
      "where": "after",
      "text": "yandexGPTid = st.sidebar.text_input('')\nyandexGPTkey = st.sidebar.text_input('')\ngigaChatKey\n= st.sidebar.text_input('')
"
    }
  },
  {
    "id": "003",
    "pattern_type": "client_init",
```

```

"comment": "Подсказка по инициализации альтернативного клиента/модели.",
"find": {
  "pattern": "client = OpenAI("
},
"insert": {
  "where": "after",
  "text": "llm = GigaChat(credentials=gigaChatKey, scope=\"GIGACHAT_API_PERS\",
model=\"GigaChat\", verify_ssl_certs=False, streaming=False)"
},
},
{
  "id": "004",
  "pattern_type": "response_extract",
  "comment": "Подсказка по получению текста ответа в другом SDK (пример схемы для GigaChat через
HumanMessage).",
  "find": {
    "pattern": "msg = response.choices[0].message.content"
  },
  "insert": {
    "where": "after",
    "text": "message = [HumanMessage(content=prompt)]\nresponse = (client(message)).content"
  }
},
{
  "id": "005",
  "pattern_type": "key_check",
  "comment": "Альтернативная проверка наличия параметров доступа для другого провайдера.",
  "find": {
    "pattern": "if not openai_api_key:"
  },
  "insert": {
    "where": "after",
    "text": "if not gigaChatKey:"
  }
}
]

```

Листинг 3.2 — report.txt:

ОТЧЁТ О ВСТАВКАХ (report.txt)

Время: 2026-01-17 13:31:53

Срабатываний (вставок) всего: 3

ID: 001

Тип: imports

Комментарий правила: Добавление импортов для GigaChat и HumanMessage (LangChain). Вставляем в начало файла.

Количество вставок: 1

Где:

- вставка: start (в начало файла)

ID: 004

Тип: response

Комментарий правила: Шаблон: находит вызов chat.completions.create и извлечение content независимо от имён переменных (\$CLIENT, \$RESP, \$MSG).

Количество вставок: 1

Где:

- совпадение: строки 26-27, вставка after (после строки 27)

ID: 005

Тип: auth

Комментарий правила: Шаблон: находит проверку ключа независимо от имени переменной (\$KEY).

Количество вставок: 1

Где:

- совпадение: строки 19-19, вставка after (после строки 19)

ПРИЛОЖЕНИЕ 4. МАТЕРИАЛЫ ДЛЯ ЗАПУСКА

В данном приложении представлены артефакты, обеспечивающие воспроизводимый запуск утилиты аннотирования и демонстрационного LLM-прототипа:

- Dockerfile — контейнер для запуска аннотатора;
- LLM_Dockerfile — контейнер для запуска преобразованного Streamlit-приложения;
- entrypoint.sh — универсальная точка входа (позволяет запускать любой `.py` внутри контейнера, передавая имя файла);
- requirements.txt — перечень зависимостей для Streamlit-прототипов;
- README — последовательность команд для сборки и запуска;
- .dockerignore** — исключение лишних файлов из контекста сборки.

Листинг 4.1 — Dockerfile

```
# Базовый образ: Python 3.11 в варианте slim.
# slim — облегчённый образ, в котором нет лишних пакетов, но есть Python.
FROM python:3.11-slim
# Рабочая директория внутри контейнера.
# Все дальнейшие операции (копирование/запуск) выполняются относительно неё.
WORKDIR /work
# Копируем внутрь контейнера только файл аннотатора.
COPY Annotator.py .
# ENTRYPOINT — команда, которая запускается при старте контейнера.
# Мы запускаем аннотатор как: python Annotator.py <rules.json> <input.py> <output.py>
ENTRYPOINT ["python", "Annotator.py"]
```

Листинг 4.2 — LLM_Dockerfile

```
# Базовый образ: Python 3.11 slim.
FROM python:3.11-slim
# Рабочая директория, в которую будет скопирован проект.
WORKDIR /app
# Установка системных пакетов.
# build-essential нужен, когда некоторые Python-зависимости требуют компиляции.
# После установки очищаем apt-кэш, чтобы образ был меньше.
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*
# Копируем requirements.txt отдельно
# Это ускоряет повторные сборки: если requirements.txt не менялся, слой с pip install берётся из кэша.
COPY requirements.txt /app/requirements.txt
# Устанавливаем зависимости Python.
# --no-cache-dir уменьшает размер итогового образа (не сохраняет кеш pip).
```

```

RUN pip install --no-cache-dir -r /app/requirements.txt
# Streamlit по умолчанию работает на порту 8501.
# EXPOSE сообщает, что контейнер “предполагает” использование этого порта.
EXPOSE 8501
# Переменные окружения Streamlit:
# headless: без открытия браузера внутри контейнера
# address 0.0.0.0: слушать все интерфейсы (иначе извне контейнера не достучаться)
# - port: порт 8501
# - gather_usage_stats=false: отключение сбора статистики
ENV STREAMLIT_SERVER_HEADLESS=true \
    STREAMLIT_SERVER_ADDRESS=0.0.0.0 \
    STREAMLIT_SERVER_PORT=8501 \
    STREAMLIT_BROWSER_GATHER_USAGE_STATS=false
# Копируем скрипт entrypoint.sh в корень контейнера.
# Он нужен, чтобы запускать streamlit с именем файла, переданным в docker run.
COPY entrypoint.sh /entrypoint.sh
# Даем права на исполнение скрипта.
RUN chmod +x /entrypoint.sh
# Копируем в контейнер весь проект (включая output.py и другие файлы).
# Сборочный контекст ограничивают через .dockerignore, чтобы не тащить лишнее.
COPY . /app
# ENTRYPOINT запускает entrypoint.sh.
# Имя приложения передается как аргумент docker run (например: output.py).
ENTRYPOINT ["/entrypoint.sh"]

```

Листинг 4.3 — entrypoint.sh

```

#!/usr/bin/env bash
# set -e — если любая команда завершится с ошибкой, скрипт немедленно завершится.
# Это полезно, чтобы контейнер не продолжал работу в частично сломанном состоянии.
set -e
# $1 — первый аргумент командной строки, переданный в контейнер.
APP="$1"
# Проверка: если имя файла не передали, выводим ошибку и завершаем работу.
if [ -z "$APP" ]; then
    echo "ERROR: Укажи имя файла: docker run IMAGE your_app.py"
    exit 1
fi
# exec заменяет текущий процесс скрипта на процесс streamlit run. Чтобы сигналы (остановка,
# перезапуск) доходили до streamlit.
# --server.address=0.0.0.0 нужно, чтобы приложение было доступно снаружи контейнера.
# --server.port=8501 фиксирует порт.
exec streamlit run "$APP" --server.address=0.0.0.0 --server.port=8501

```

Листинг 4.4 — requirements.txt

```

streamlit # UI-обвязка для быстрого создания веб-интерфейсов на Python
openai # библиотека для доступа к OpenAI API (используется в исходных примерах/прототипах)
ddgs # библиотека для запросов к DuckDuckGo (популярна в проектах с интернет-поиском)
anthropic # библиотека для доступа к моделям Anthropic (в примерах встречается как провайдер)
trubrics # библиотека для сбора обратной связи/оценок (используется в некоторых UI-примерах)
streamlit-feedback # компонент обратной связи в Streamlit
langchain-community # набор интеграций LangChain, вынесенных в отдельный пакет
gigachain-community # совместимые компоненты/интеграции для работы с GigaChat

```

Листинг 4.5 — README

Примечание: в данном README используются фигурные скобки {...} как обозначение того, что пользователь подставляет свои имена образов и файлов.

для запуска преобразователя:

```
docker build -t {project-updater} .
```

```
docker run --rm -v "${PWD}:/data" -w /data {project-updater} {rules}.json {ProgramForConversion}.py  
{output}.py
```

для запуска проекта:

```
docker build -t {my-streamlit} .
```

```
docker run --rm -p 8501:8501 -v "${PWD}:/app" {my-streamlit} {output}.py
```

Листинг 4.6 — .dockerignore

__pycache__/ — кеш Python скомпилированные байткод-файлы, не нужен в образе.

.рус — отдельные скомпилированные файлы Python, также не нужны.

.venv/, — виртуальные окружения, внутри контейнера зависимости ставятся заново через pip, поэтому переносить окружение бессмысленно.

venv/

.git/ — история репозитория не нужна для запуска приложения.

.idea/ — настройки IDE (PyCharm/IDEA), не нужны в контейнере.